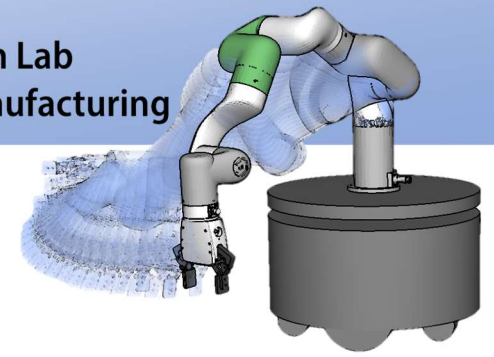


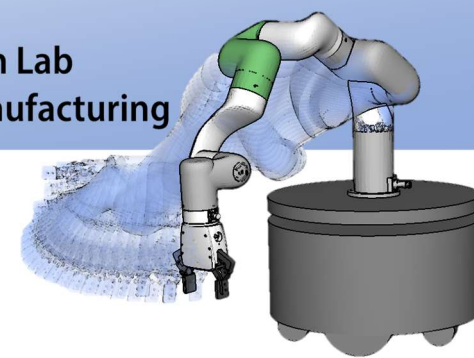


スタック・キュー・ 再帰・リスト

原田研究室 准教授 万 偉偉

基礎工学部 F522号室

wan@sys.es.osaka-u.ac.jp



宿題について

- コマンドライン引数

- `int main(int argc, char *argv[])` と記述する
- `int argc`: 引数の総個数（プログラム名も含む）
- `char *argv[]`: 引数の文字列を指す ポインタの配列 を表します

`argc = 3`

`argv[0]` 'a.exe'

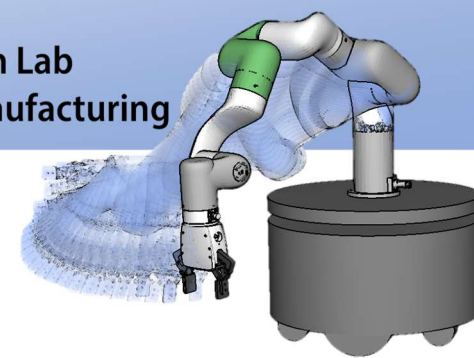
`argv[1]` 'x'

`argv[2]` 'y'

a.exe

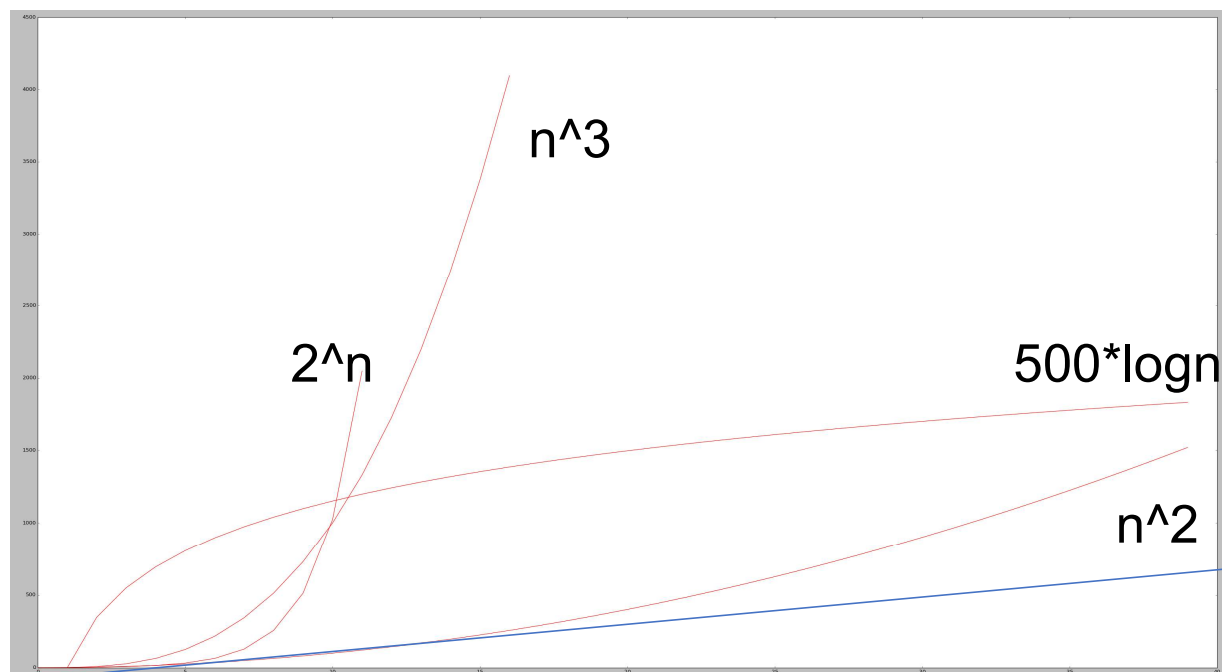
```
8  #include <stdio.h>
9  int main( int argc, char * argv [] ) {
10     printf( "argc = %d\n", argc );
11     for( int i = 0; i < argc; ++i )
12     {
13         printf( "argv[ %d ] = %s\n", i, argv[ i ] );
14     }
15 }
16
17
```

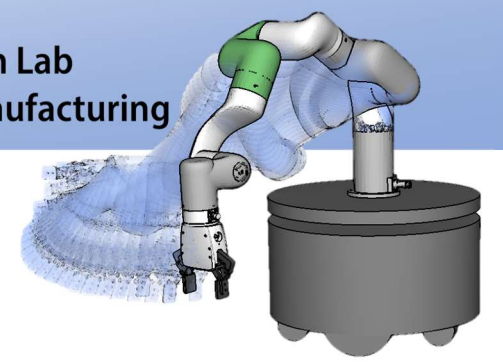
```
argc = 1
argv[ 0 ] = /home/a.out
```



復習

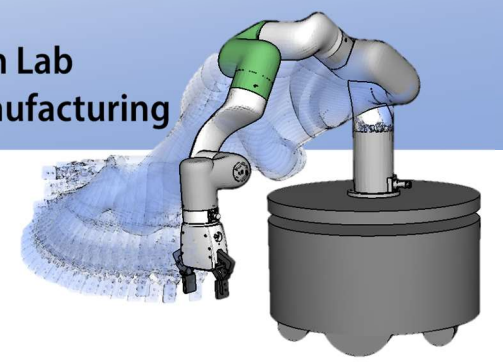
- 計算量
- 計算量の計算
 - 最大次数の項以外を除く
 - 係数を除く
- 最大次数の項
 - 2^n
 - n^3
 - n^2
 - $N \log n$
 - n
 - $\log n$





復習

- 初等的整列
- ある基準(<) でデータを並べ替えること
 - 基準(<)は各自が定義する.
- 安定な整列
- 選択ソート
 - 未整列部分から最小要素を発見し, 整列部分の後の要素とスワップ.
- バブルソート
 - 後から近接要素のみスワップ. これの2回繰り返す.
- 挿入ソート
 - 未整列部分の先頭要素を, 整列部分の適切な位置に挿入



復習

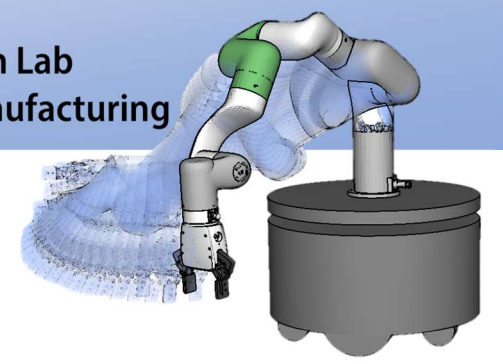
- 選択ソート

```
for (i ← 0..n-2)
  lowest ← argmini < j < n (a[j])
  swap(a[i], a[lowest])
```



配列 $a[]$ の i 以降の要素
で最小値の要素番号を求める

- ループ n 回, argmin の計算量 $O(n)$
- 選択ソートの計算量は $O(n^2)$

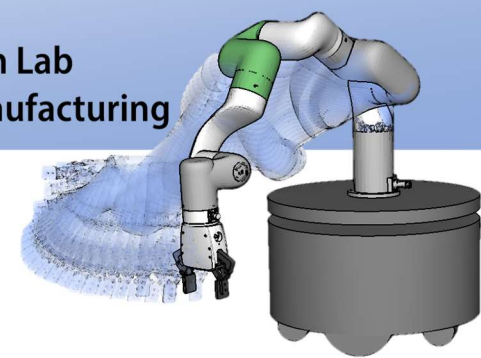


復習

• バブルソート

```
for (i ← 0..n-1)
  for (j ← n-1..i)
    if !(a[j-1] ≤ a[j]) swap(a[j-1], a[j])
```

右辺値の交換



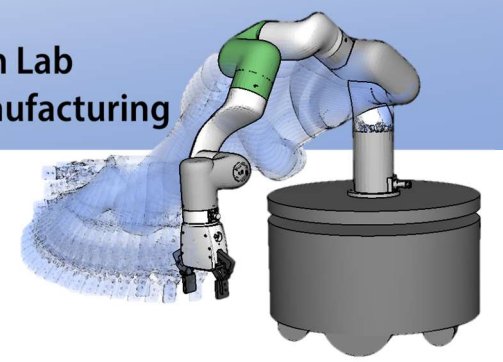
復習

• 挿入ソート

```
for (i ← 1..n)
  j ← i
  while (j >= 1 && ! (a[j-1] ≤ a[j]))
    swap( a[j-1], a[j] )
    j ← j-1
  endwhile
endfor
```

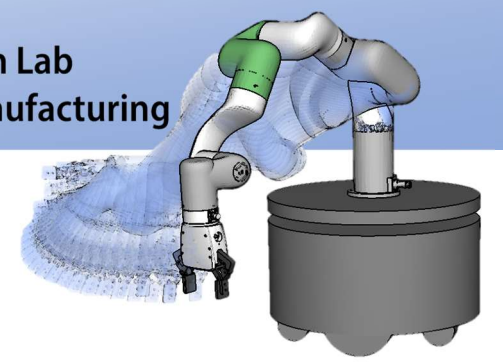
計算量：外側ループ $O(n)$
 内側ループ $O(n)$

合計： $O(n^2)$



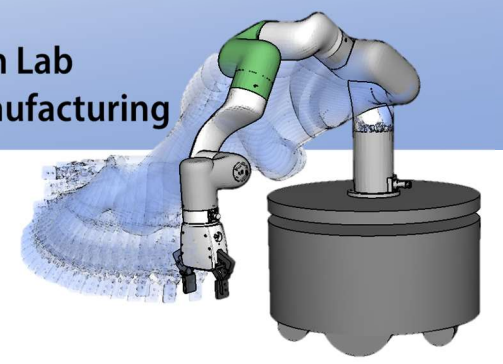
復習

- 線形探索法(linear search)
 - データを最初から順番に探索する
 - 例) {2, 4, 5, 8, 9, 11, 6, 7, 15, 20} から15を探す
最初の要素から始めて9回目→計算量 $O(n)$
- 二分探索法(binary search)
 - データをあらかじめソートしておき, 中央の要素から検索する.

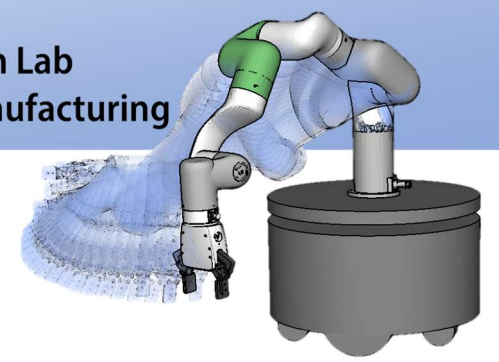


今日の内容

- スタック
- キュー
- 再帰
- リスト

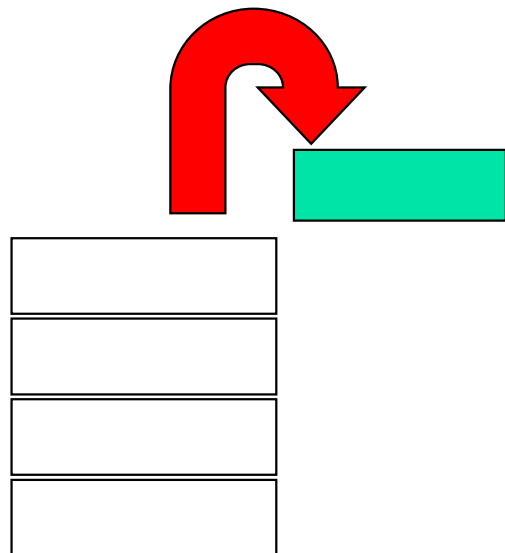


スタック



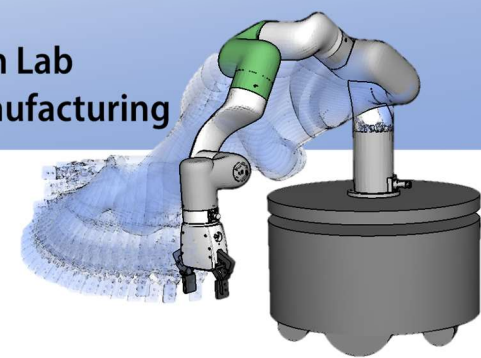
ADT Stack (スタック)

- データの集まりキュー
- 一番上の要素しか操作できない
- イメージ的にはものを上に積んだ状態



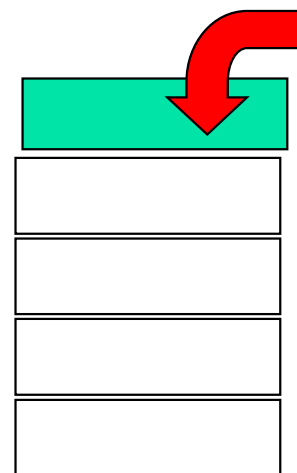
スタックに可能な操作

- 一番上の要素の値を見る(top)
- 一番上の要素を取り除く(pop)
- 上に要素を積む(push)
- スタックが空？(isEmpty)
- スタックの要素数(size)

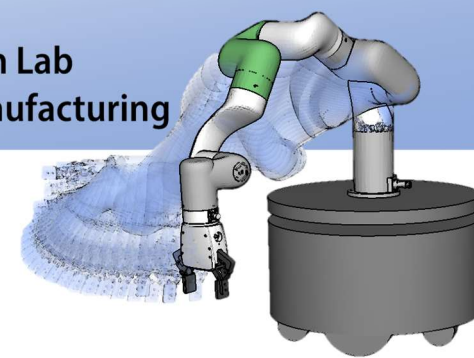


ADT Stack

- Push (プッシュ操作)
 - スタックの上に要素を積む操作
 - スタックの要素数は+1される

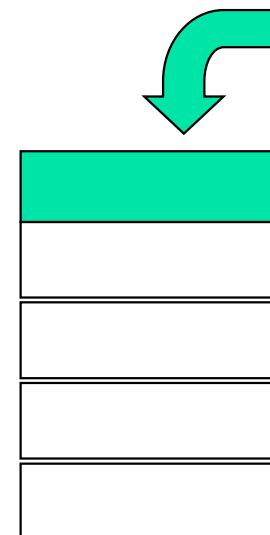


要素数 = 4

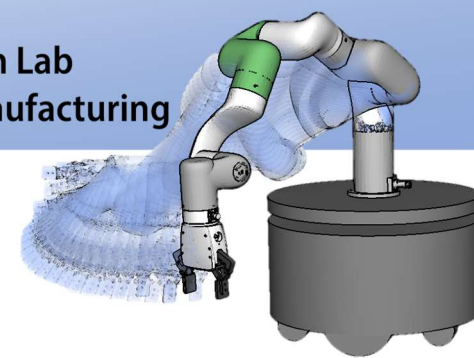


ADT Stack

- Push (プッシュ操作)
 - スタックの上に要素を積む操作
 - スタックの要素数は+1される

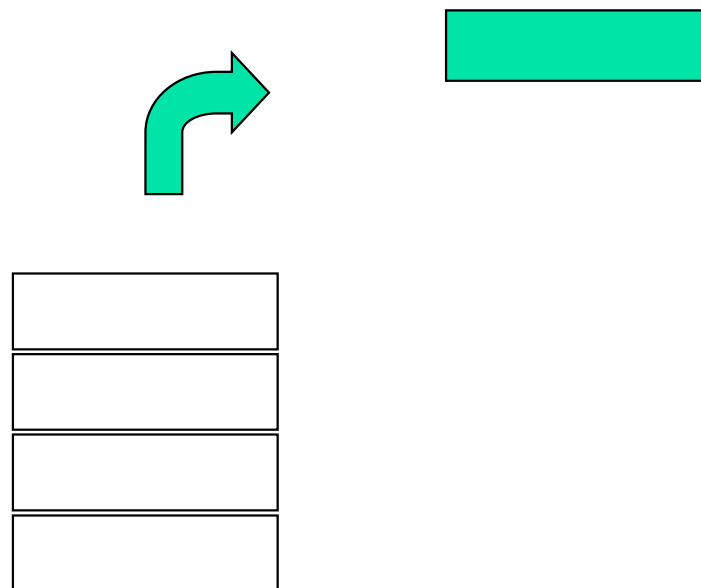


要素数 = 5

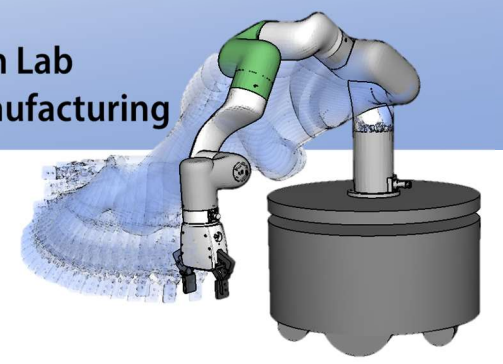


ADT Stack

- Pop (ポップ操作)
 - スタックの上から要素を取り除く操作
 - スタックの要素数は-1される

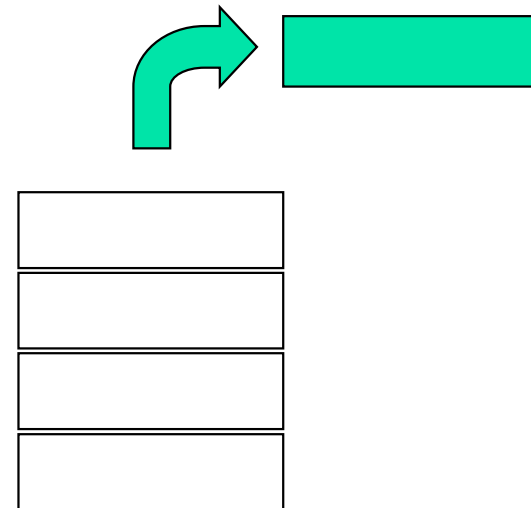


要素数 = 5

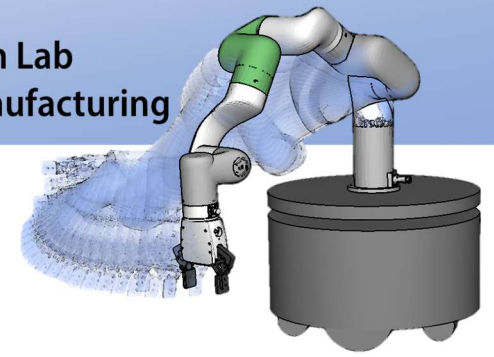


ADT Stack

- Pop (ポップ操作)
 - スタックの上から要素を取り除く操作
 - スタックの要素数は-1される

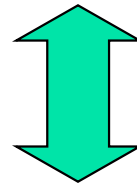


要素数 = 4

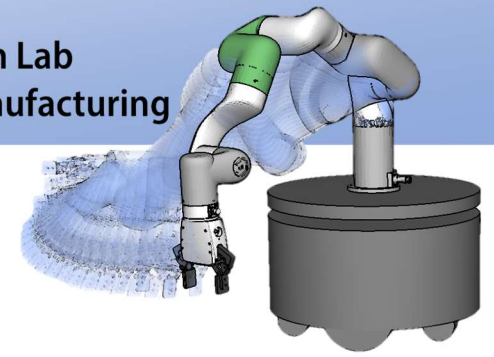


LIFO (Last-in first-out)

- スタックの要素の出し入れの特性を表している
- 最後に加えた要素が，最初に取り出される



FIFO (First-in first-out)



配列による ADT Stack の実現

- 出し入れする要素型（任意の型）の配列で実現
- 現在の要素数を保持するカウンタ N 利用
- 配列から要素が溢れないように最大容量 MAX を定義

スタックの最大容量 →

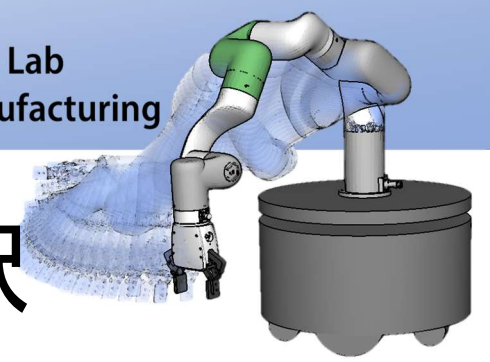
スタックの top →

stack[$MAX-1$]	
...	
stack[N]	
stack[$N-1$]	
...	
stack[3]	
stack[2]	
stack[1]	
stack[0]	

疑似コードによる ADT Stack の実装

```

Push: stack[ $N++$ ] ← data if  $N < MAX$ 
Pop: data ← stack[ $--N$ ] if  $N > 0$ 
Top: stack[ $N-1$ ] if  $N > 0$ 
isEmpty: true if  $N \leq 0$ 
         otherwise false
Size:  $N$ 
    
```



疑似コードから C 言語への翻訳

ここでは、要素型を element とする.

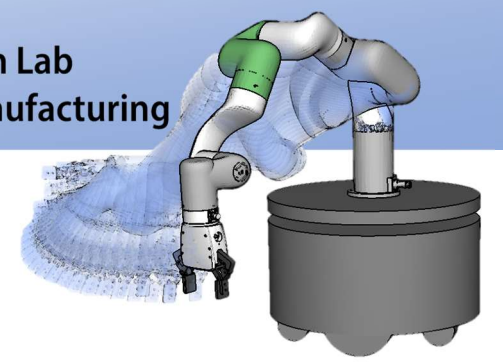
Push: $\text{stack}[\text{N}++] \leftarrow \text{data}$ if $\text{N} < \text{MAX}$

```
void Push(element data) {  
    if ( $\text{N} < \text{MAX}$ )  $\text{stack}[\text{N}++] = \text{data}$ ;  
    /*  $\text{N} \geq \text{MAX}$  の時の動作は設計する必要あり */  
}
```

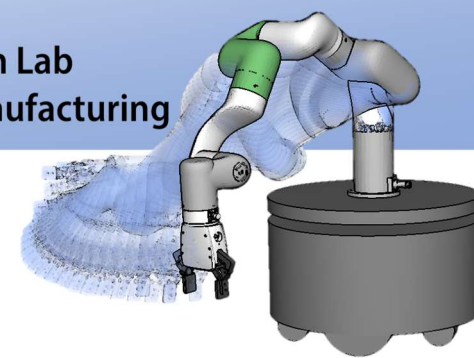
isEmpty: true if $\text{N} \leq 0$, otherwise false

```
int isEmpty() {  
    if ( $\text{N} \leq 0$ ) return !0; else return 0;  
}
```

C言語の偽(false)は 0
真(true)は !0

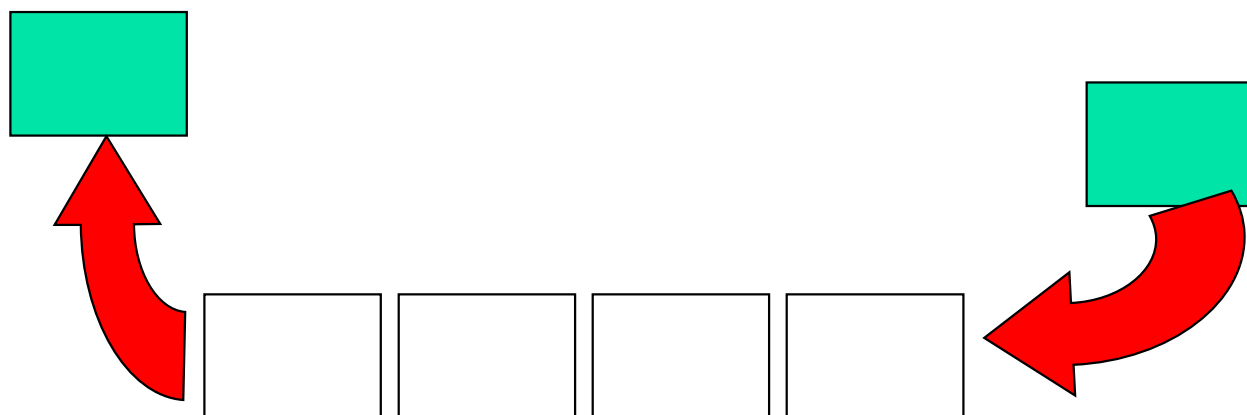


待ち行列（キュー）

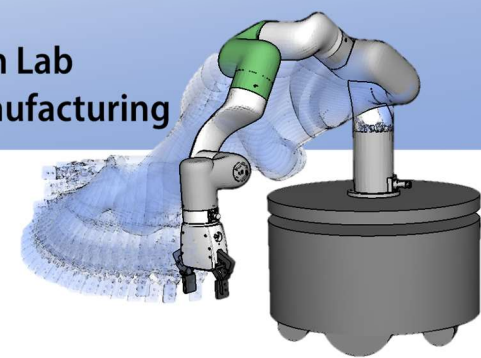


ADT Queue(待ち行列)

- データの集まり
- 要素の挿入，削除が端でしかできない
- イメージ的には人の待ち行列

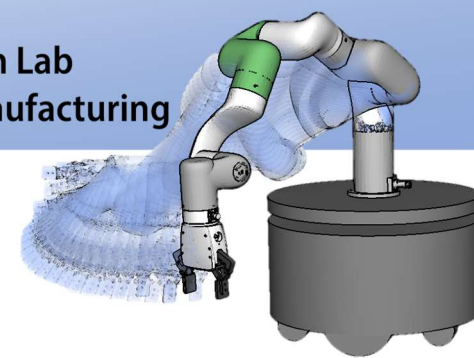


待ち行列の先頭



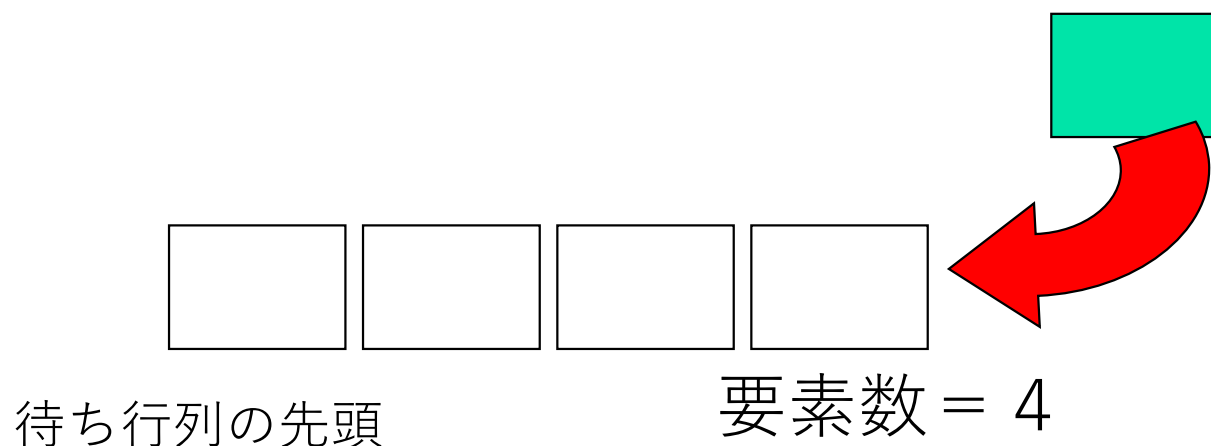
ADT Queue の操作

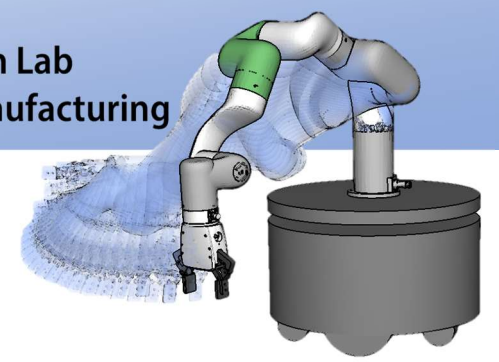
- 待ち行列に加える(enqueue)
- 待ち行列から取り出す (dequeue)
- 待ち行列の要素数(size)
- 先頭の要素を見る(front)
- 待ち行列が空？ (isEmpty)



ADT Queue

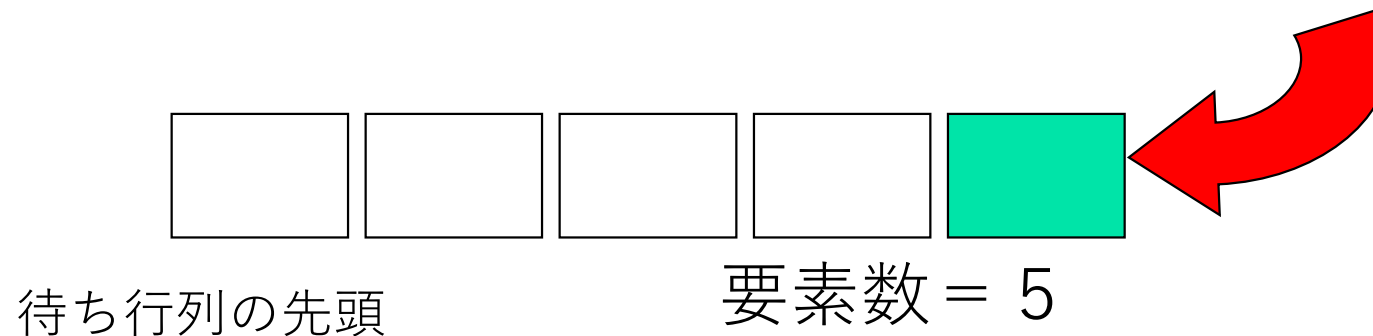
- Enqueue
 - 待ち行列の最後尾に要素を加える
 - 待ち行列の要素数は+1される

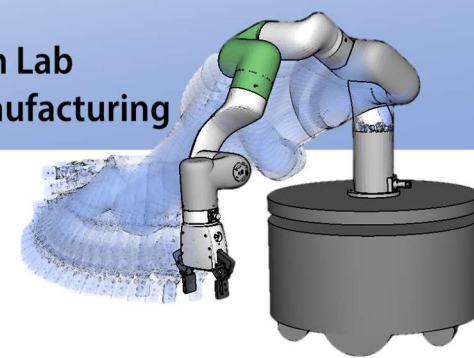




ADT Queue

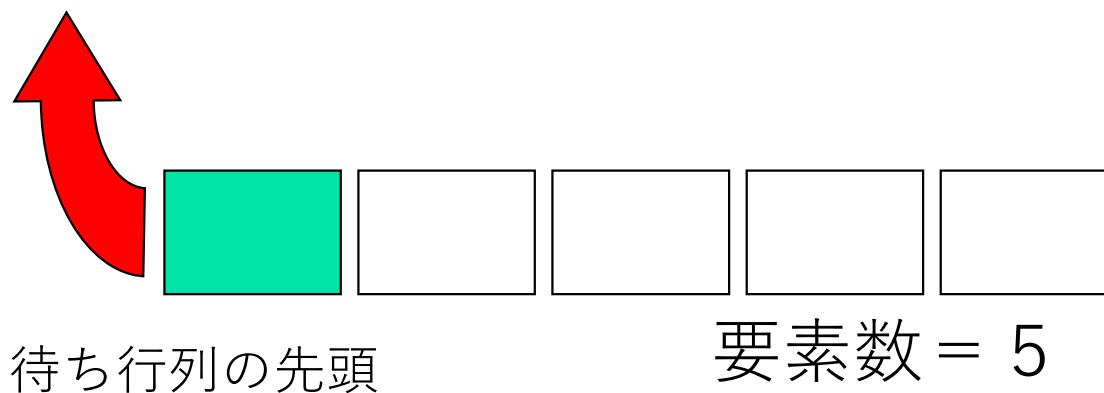
- Enqueue
 - 待ち行列の最後尾に要素を加える
 - 待ち行列の要素数は+1される

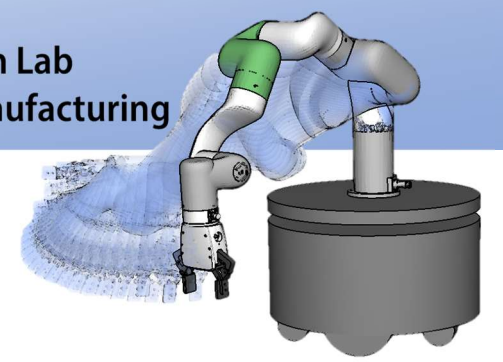




ADT Queue

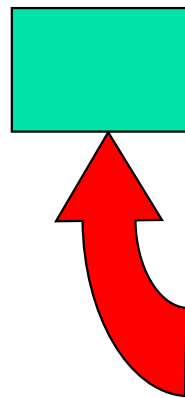
- Dequeue
 - 待ち行列の先頭から要素を取り出す
 - 待ち行列の要素数は-1される





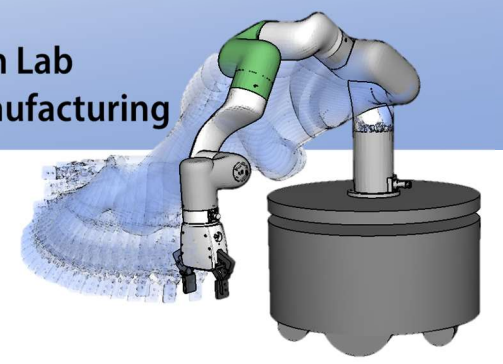
ADT Queue

- Dequeue
 - 待ち行列の先頭から要素を取り出す
 - 待ち行列の要素数は-1される



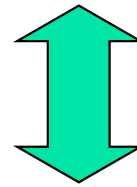
待ち行列の先頭

要素数 = 4

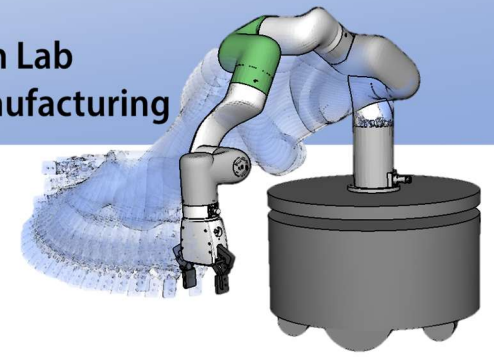


FIFO (First-in first-out)

- 待ち行列の要素の出し入れの特性を表している
- 最初に加えた要素が最初に取り出される



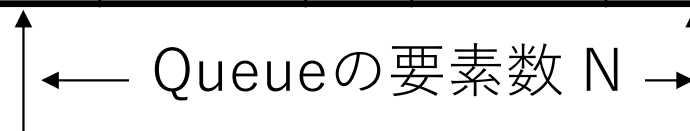
LIFO (Last-in first-out)



配列によるADT Queueの実現

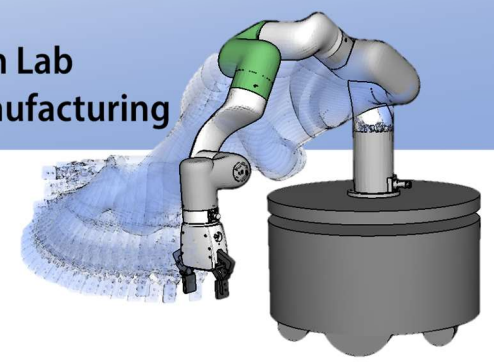
- 出し入れする要素型の配列で実現
- 待ち行列の先頭と最後尾の要素位置（配列の添え字）を格納するインデックス F , R を利用
- 配列から要素が溢れないように配列の最大容量 MAX を定義
- 現在の要素数を保持するカウンタ N 利用

$Q[0]$	$Q[1]$	...	$Q[F]$	$Q[F+1]$	...	$Q[R-1]$	$Q[R]$	...	$Q[MAX-1]$


 Queueの要素数 N

Queueの先頭

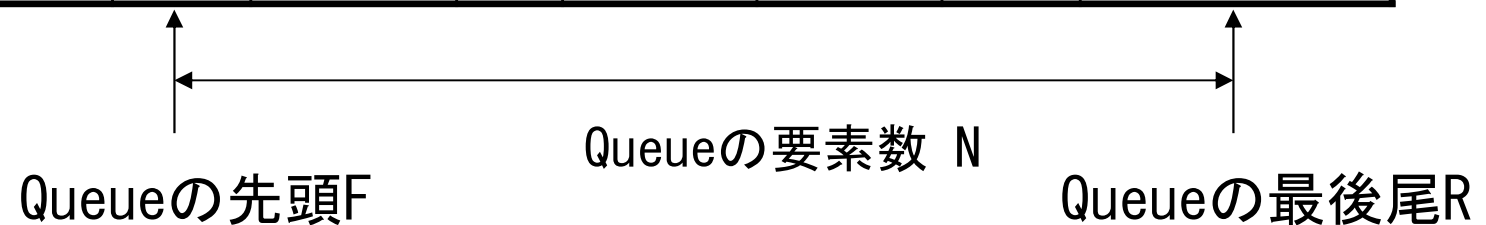
Queueの最後尾

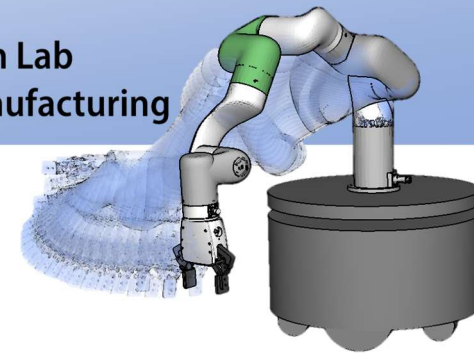


配列によるADT Queueの実現

- F , R は要素が出し入れされると+1される
- F または R が MAX 以上になったとき, 0 に設定して配列を再利用する

$Q[0]$	$Q[1]$	...	$Q[F]$	$Q[F+1]$	...			...	$Q[MAX-1]$

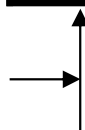




配列によるADT Queueの実現

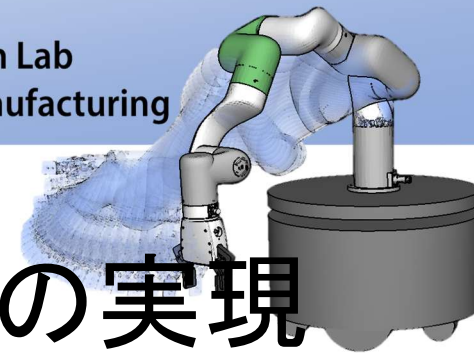
- F, R は要素が出し入れされると+1される
- F または R が MAX以上になったとき, 0に設定して配列を再利用する
- 要素数NがMAXを越えることはできない

Q[0]	Q[1]	...	Q[F]	Q[F+1]	...			...	Q[MAX-1]



Queueの要素数 N

Queueの最後尾R Queueの先頭F



疑似コードによるADT Queueの実現

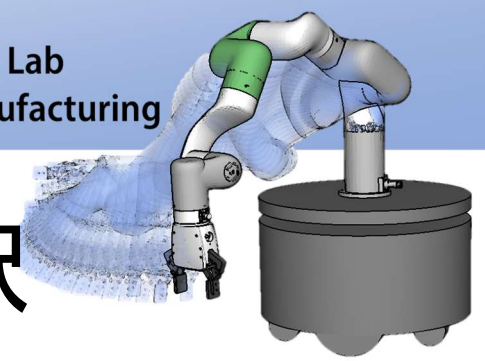
Enqueue: $Q[R++] \leftarrow \text{data}$ if $N < \text{MAX}$
 $N \leftarrow N+1$
 $R \leftarrow 0$ if $R \geq \text{MAX}$

Dequeue: $\text{data} \leftarrow Q[F++]$ if $N > 0$
 $N \leftarrow N-1$
 $F \leftarrow 0$ if $F \geq \text{MAX}$

Front: $Q[F]$ if $N > 0$

isEmpty: true if $N \leq 0$
otherwise false

Size: N



疑似コードから C 言語への翻訳

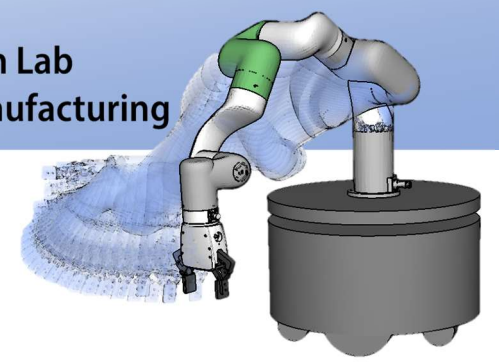
ここでは、要素型を element とする

Enqueue: $Q[R++] \leftarrow \text{data}$ if $N < \text{MAX}$

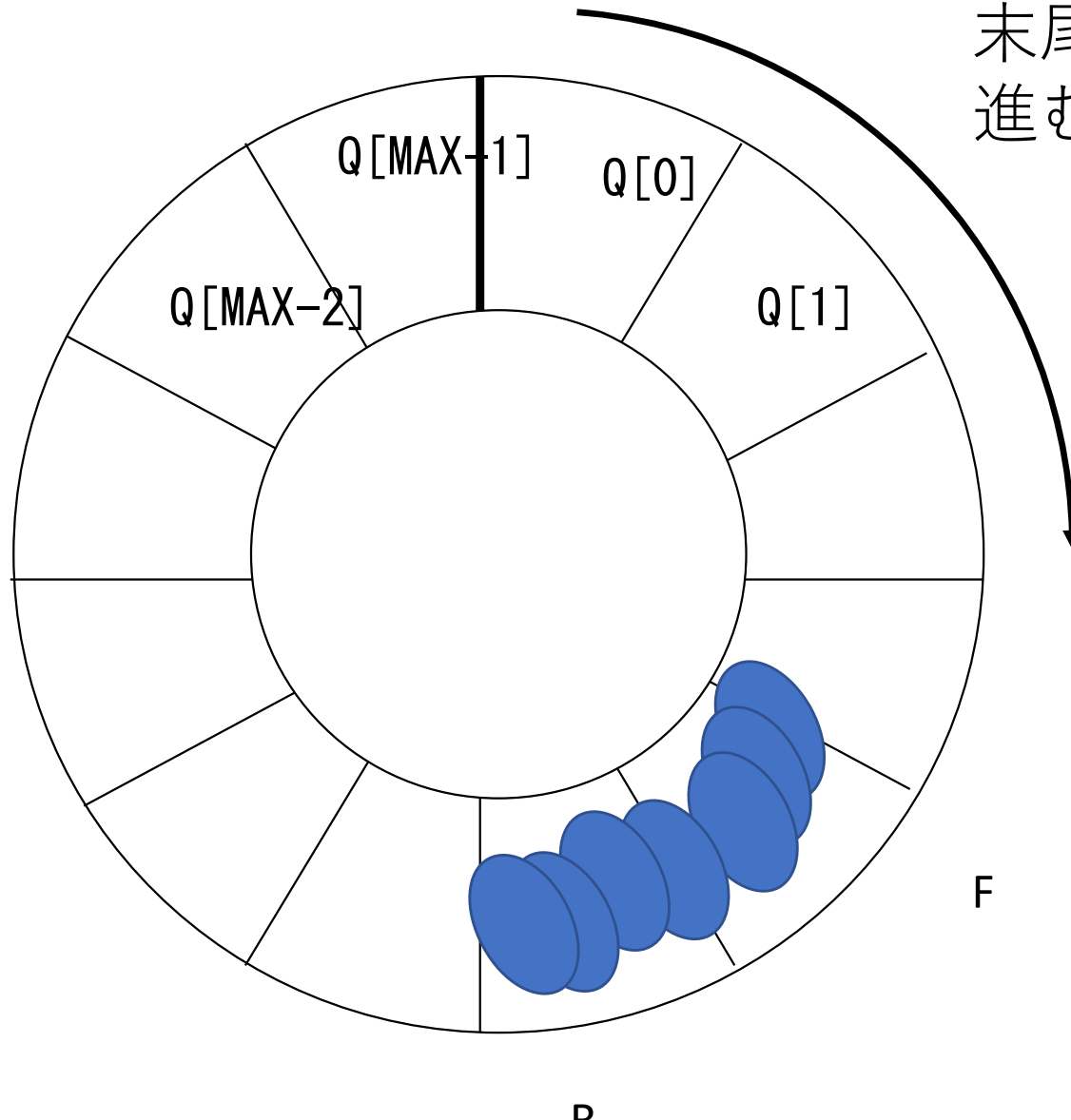
$N \leftarrow N + 1$

$R \leftarrow 0$ if $R \geq \text{MAX}$

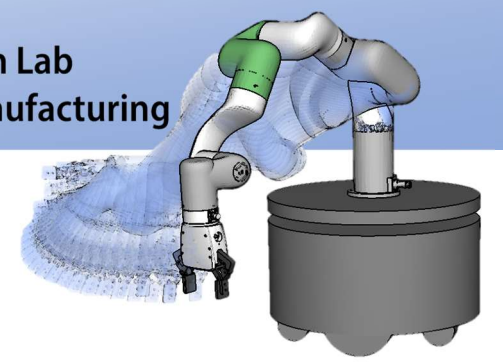
```
void Enqueue( element data ) {  
    if (N < MAX) { /* N >= MAX の時の動作は設計する必要あり */  
        Q[R++] = data;  
        N = N + 1;  
        if (R >= MAX) R = 0;  
    }  
}
```



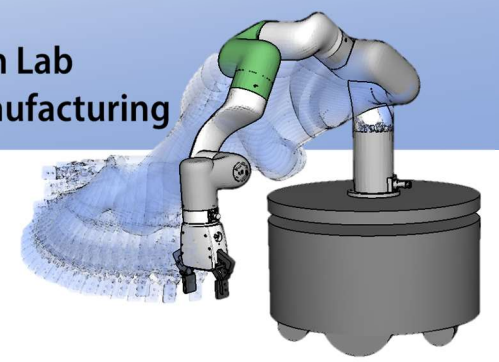
リングバッファ



末尾までカウンタが進むと先頭まで戻る

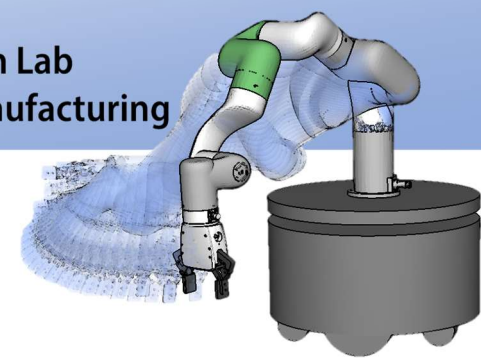


再帰



再帰

- 再帰的(recursive)な構造とは、自分自身(n 次)を定義するのに、自分自身より1次低い集合($n-1$ 次)を用い、さらにその部分集合は、より低次の部分集合を用いて定義するということを繰り返す構造. このような構造を一般的に再帰(recursion)と呼ぶ.



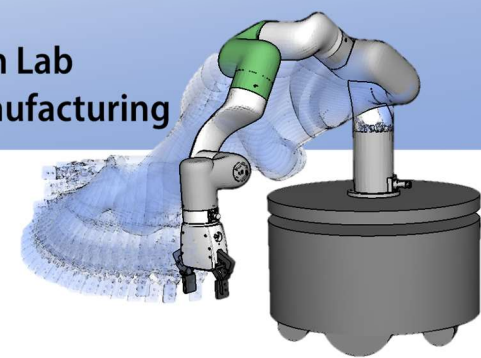
再帰

- 数学ではいろいろな概念を再帰的に定義している.

例) 数列 $\{a_1, a_2, \dots\}$

$$a_1 = 1$$

$$a_n = a_{n-1} + 2n - 1$$



再帰呼び出し

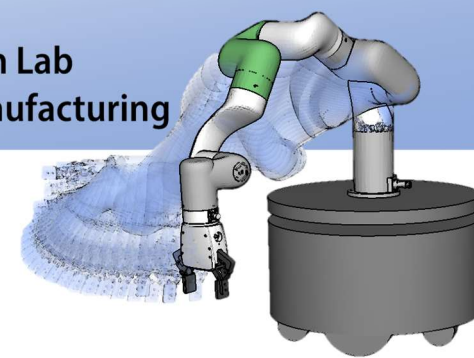
- 関数の中で自分自身を関数として呼び出すこと

例) 数列 $\{a_1, a_2, \dots\}$

$$a_1 = 1$$

$$a_n = a_{n-1} + 2n - 1$$

```
int A(int n) {  
    if (n == 1) return 1; /* 再帰呼び出しの停止条件 */  
    else return A(n-1)+2*n-1;  
}
```



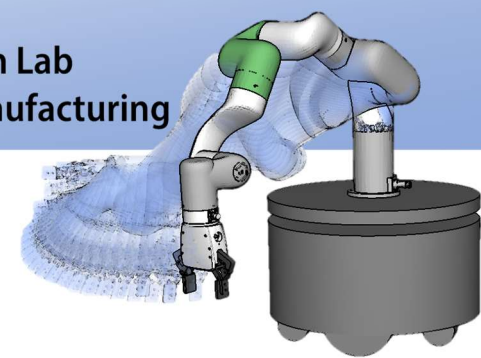
再帰呼び出し例

- 階乗($n!$)の計算プログラム

$$f_0 = 1$$

$$f_n = n \times f_{n-1}$$

```
#include <stdio.h>
int factorial(int n) {
    if (n == 0) return 1;
    else return n*factorial(n-1);
}
int main() {
    int n;
    for (n=0; n<10; ++n)
        printf( “ %2d! = %d¥n” , n, factorial(n) );
    return 0;
}
```



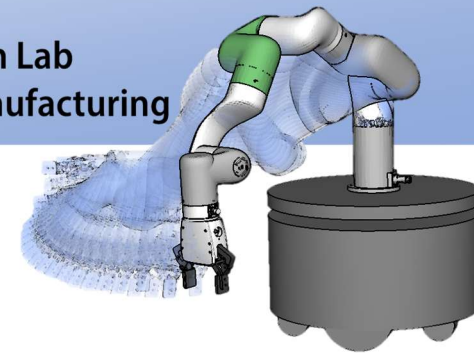
フィボナッチ数列

- フィボナッチ数列を再帰を用いて求める

$$f_1 = f_2 = 1$$

$$f_n = f_{n-1} + f_{n-2}$$

```
#include <stdio.h>
int fibonacci(int n) {
    if (n == 1 || n == 2) return 1;
    else return fibonacci(n-1)+fibonacci(n-2);
}
int main() {
    int n;
    for (n=0; n<20; ++n)
        printf( " %2d! = %d¥n" , n, fibonacci(n) );
    return 0;
}
```



再帰の利用されるところ

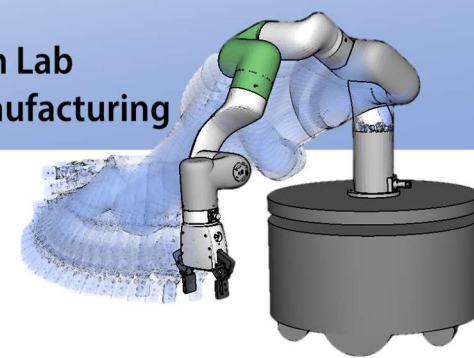
- 漸化式

$${}_nC_r = \frac{n!}{r!(n-r)!} \quad \begin{cases} {}nC_r = \frac{n-r+1}{r} {}nC_{r-1} \\ {}nC_0 = 1 \end{cases}$$

- 1回目の授業の例

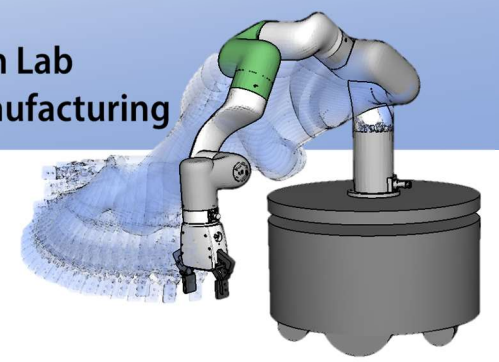
- 分割統治

- 次数を下げて問題の規模を小さくして解きやすくする.
- マージソート, 木のなぞり (今後の講義で紹介)



再帰呼び出しの展開

- 再帰呼び出しは，分かりやすいプログラムを作成することができる．
- 再帰呼び出しは，計算量がかかることがあるので，十分に検討する必要がある．
- 再帰が時間がかかる場合は，再帰呼び出しをループに展開する方法がある．
- また，自分でスタックを作成して，再帰呼び出しを構成する方法もある．



再帰で呼び出しの展開

• フィボナッチ数列 計算量は？

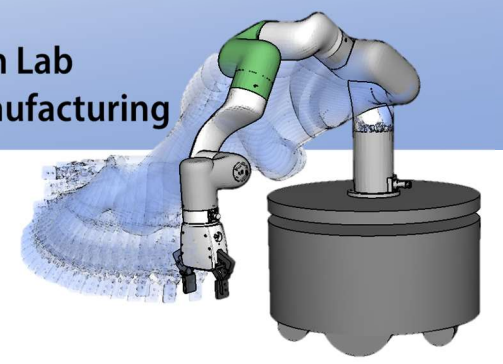
```
int fib(int n) {  
    if (n <= 1)  
        return n;  
    else  
        return fib(n - 1) + fib(n - 2);  
}
```

```
int fib(int n) {  
    int a = 0, b = 1, temp, i;  
    for (i = 2; i <= n; i++) {  
        temp = a + b;  
        a = b;  
        b = temp;  
    }  
    return (n == 0) ? a : b;  
}
```

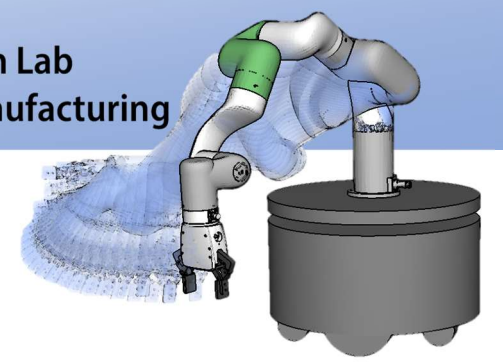
• 一般的な数列 $a_n = f(a_{n-1})$

```
int a(int n) {  
    if (n == 0) return 初期値;  
    return f(a(n - 1));  
}
```

```
int a(int n) {  
    int val = 初期値;  
    for (int i = 1; i <= n; i++) {val = f(val);}   
    return val;  
}
```



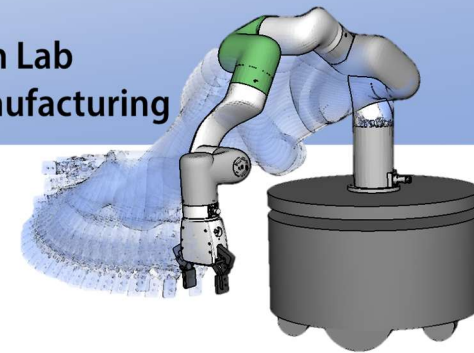
リスト構造



リスト構造

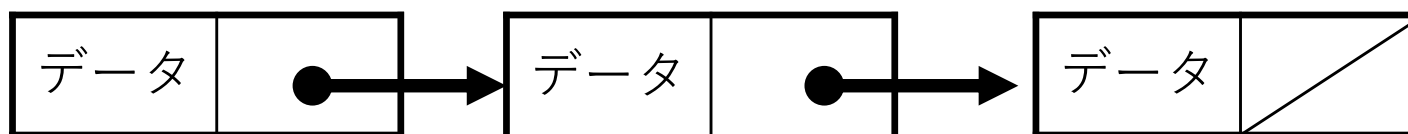
- 連結リスト
- 循環リスト
- 双方向リスト

データをポインタによってつなげたもの

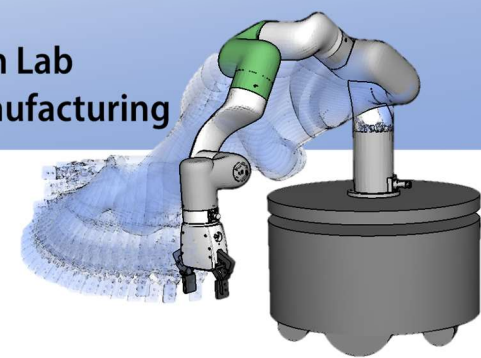


連結リスト(Linked list)

- データの集まり
- データが一覧表（リスト）のように連なっている構造
- 連なりの表現には効率のため、ポインタが用いられることが多い
- もちろん、配列とインデックスでも実装できる（リスト容量が制限）

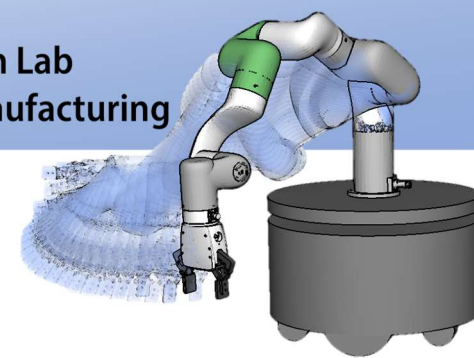


DATA[0]	DATA[1]	DATA[2]	...	DATA[N]
データ	未使用	データ	...	データ
IDX[0]	IDX[1]	IDX[2]	...	IDX[N]
2	-1	N	...	EOD



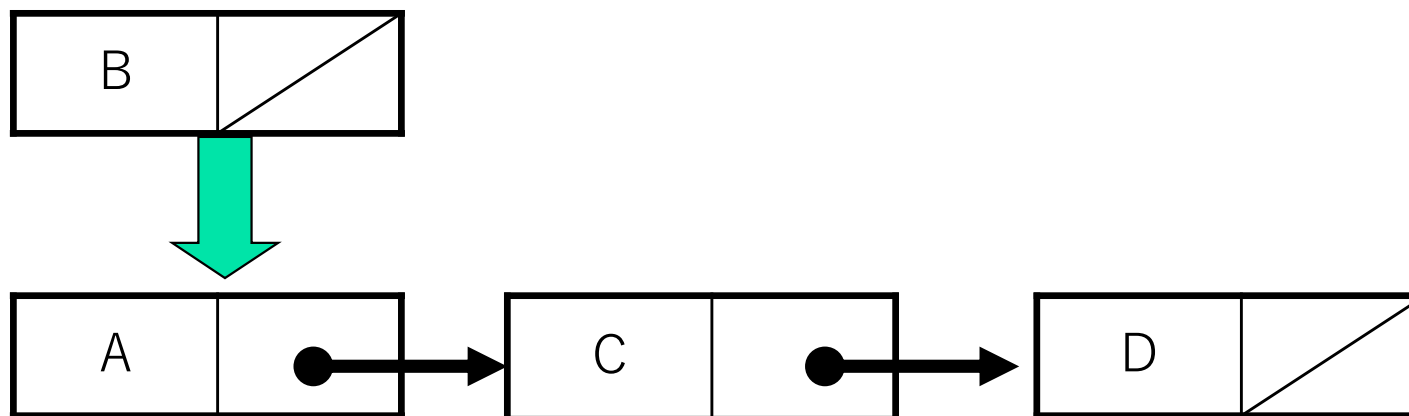
連結リストの操作

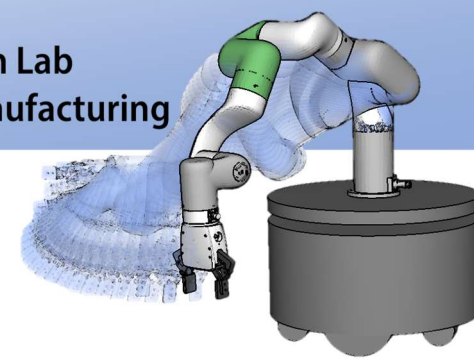
- 要素の挿入 (insert)
- 要素の削除 (erase)
- リストの連結 (concatenate)
- リストの分断 (split)
- 要素の探索 (find)
- 空リストの生成 (create)
- 要素数 (size)
- 先頭要素を見る (front)
- 最後尾要素を見る (tail)



連結リストの操作

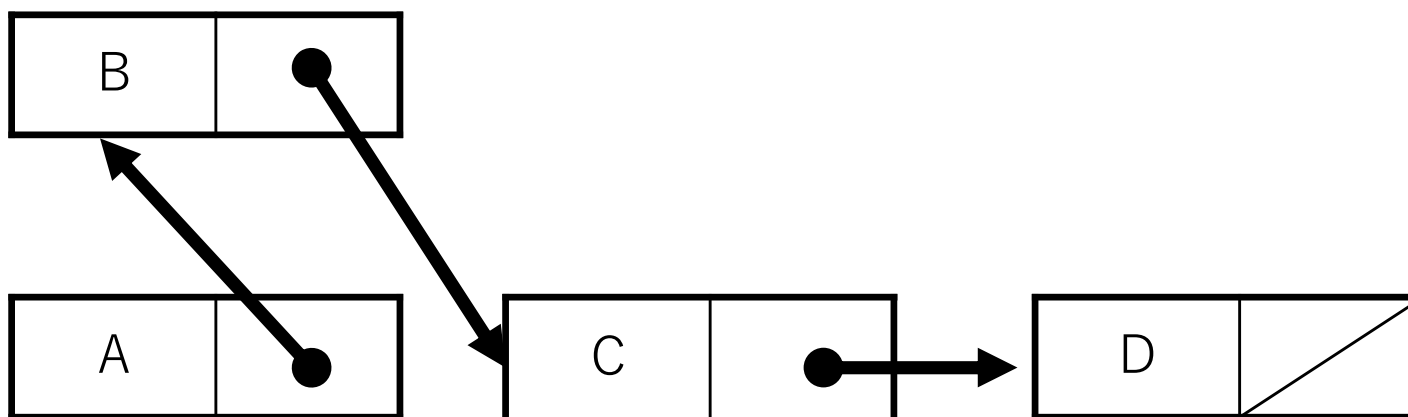
- 要素の挿入 (insert)
 - 指定された要素の次の位置に要素を挿入する

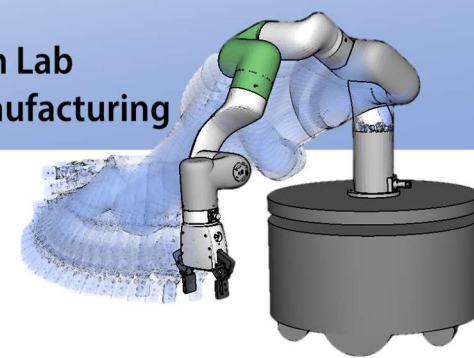




連結リストの操作

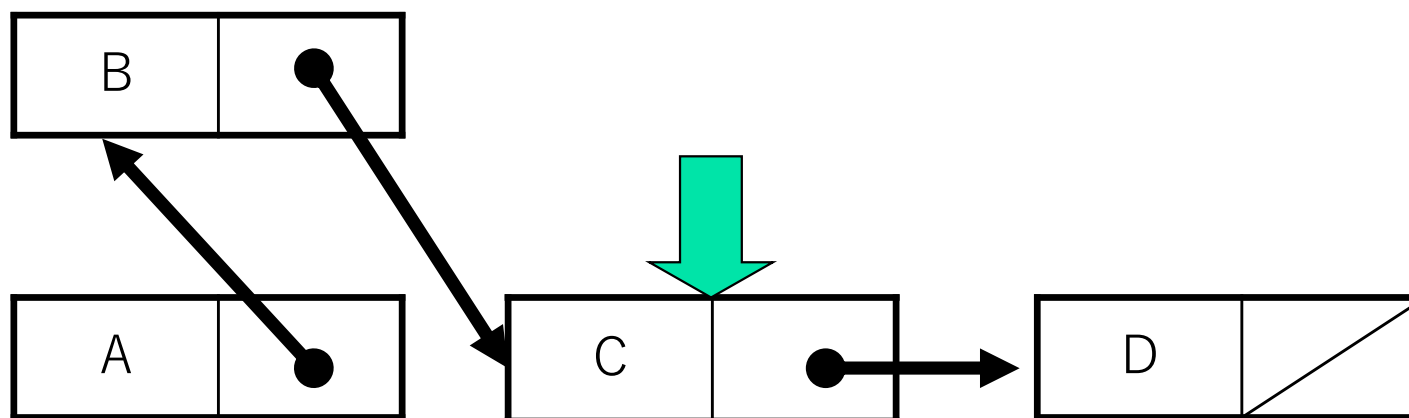
- 要素の挿入 (insert)
 - 指定された要素の次の位置に要素を挿入する

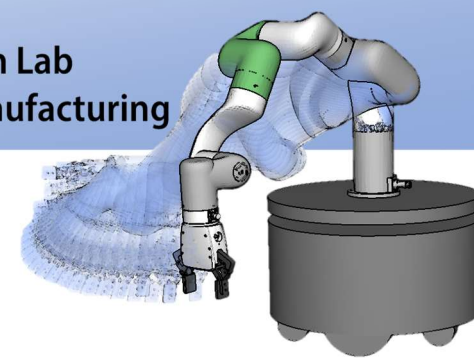




連結リストの操作

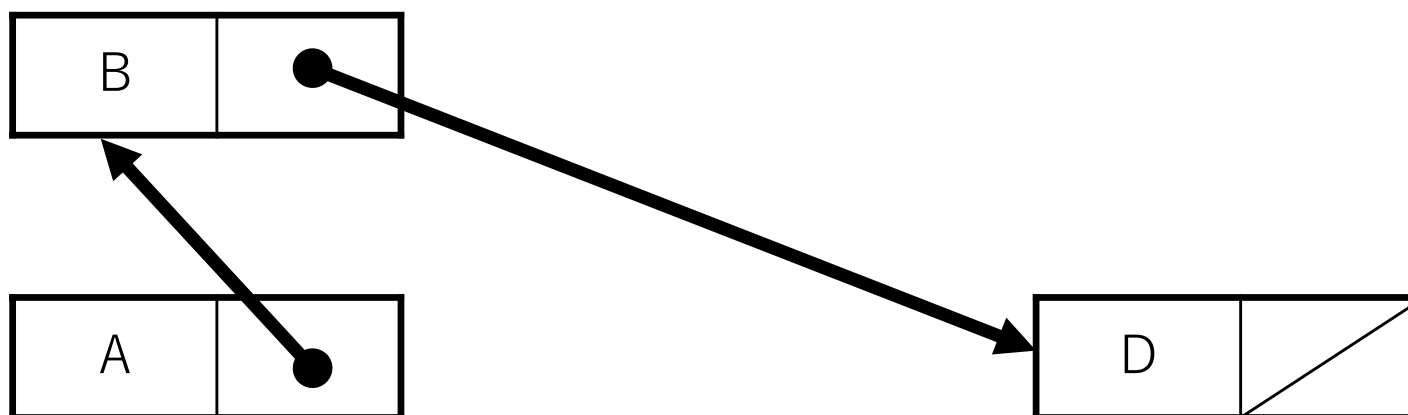
- 要素の削除 (erase)
 - 指定された要素を削除する

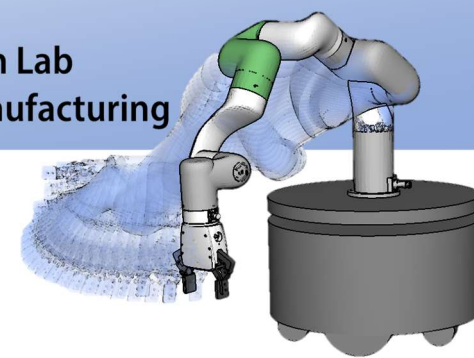




連結リストの操作

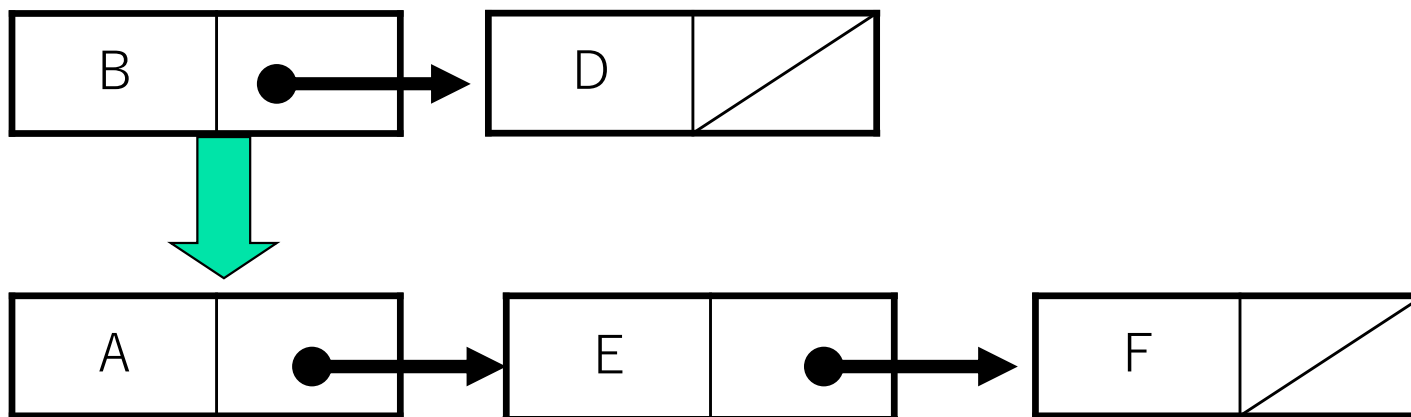
- 要素の削除 (erase)
 - 指定された要素を削除する

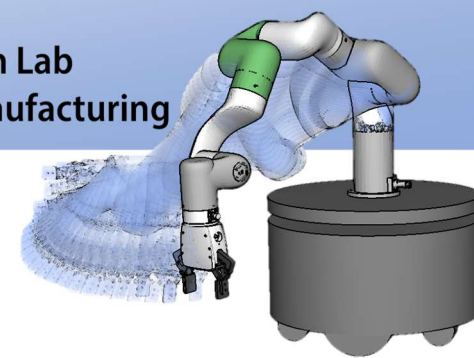




連結リストの操作

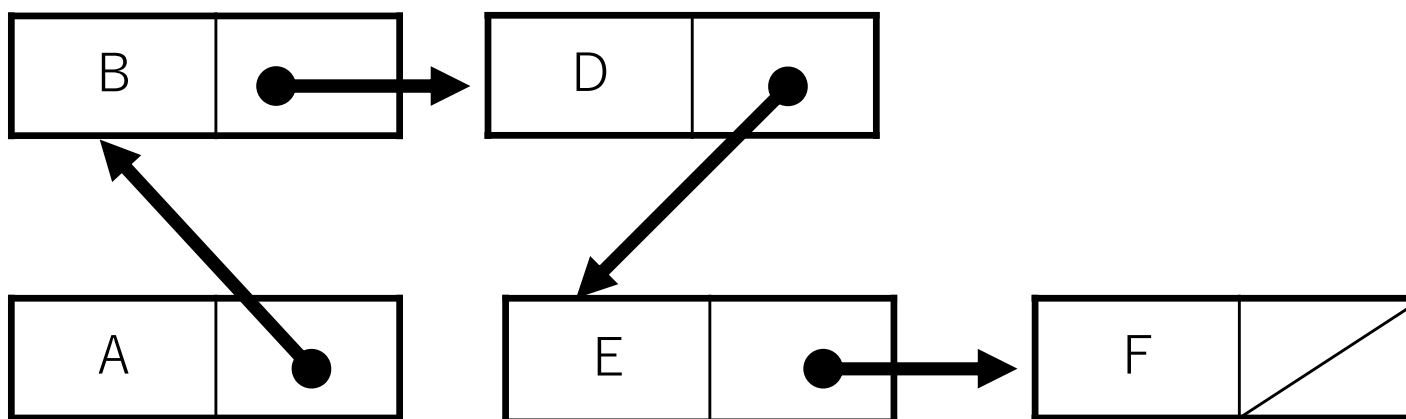
- リストの連結 (concatenate)
 - 指定された要素の次の位置にリストを連結する

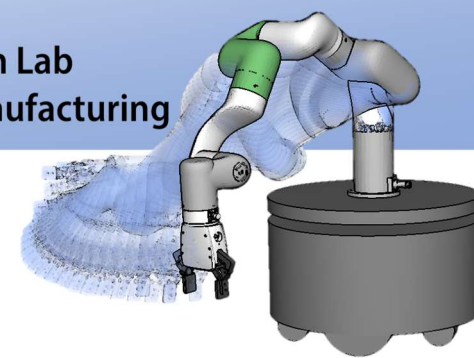




連結リストの操作

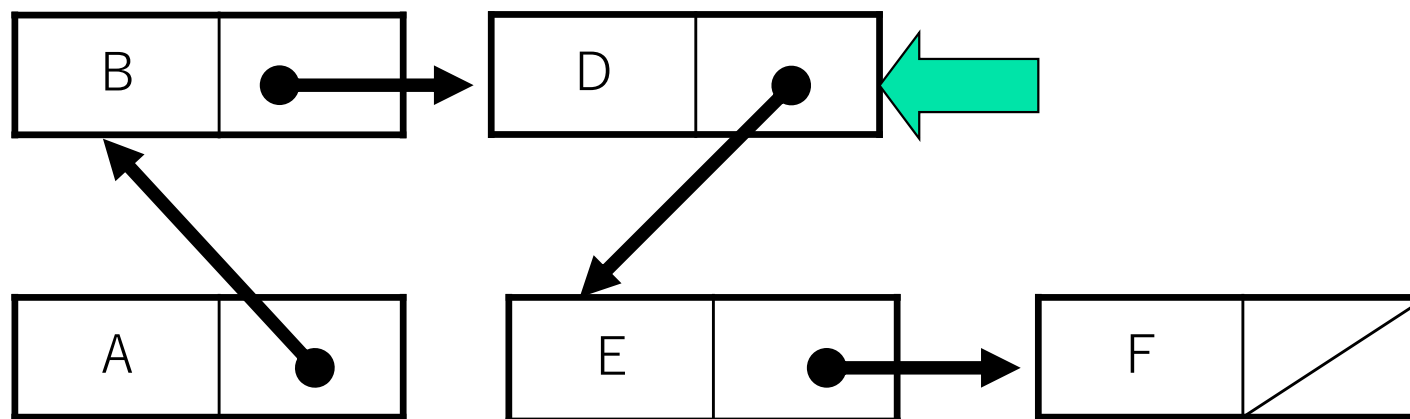
- リストの連結 (concatenate)
 - 指定された要素の次の位置にリストを連結する

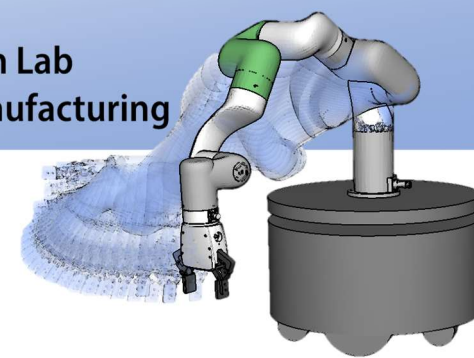




連結リストの操作

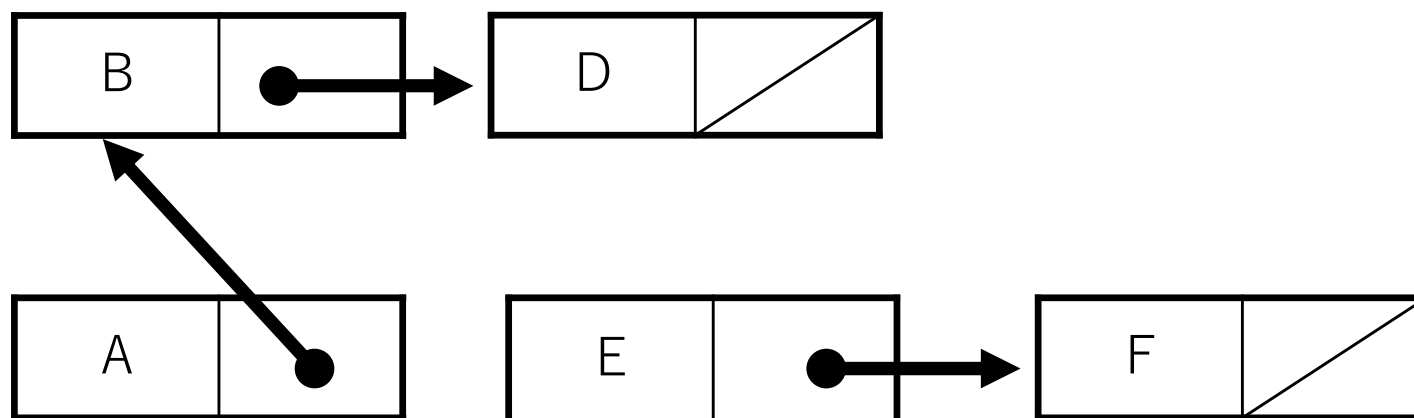
- リストの分断 (split)
 - 指定された要素の次の位置からリストを分断する

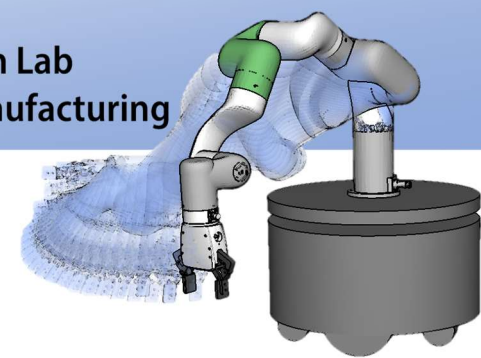




連結リストの操作

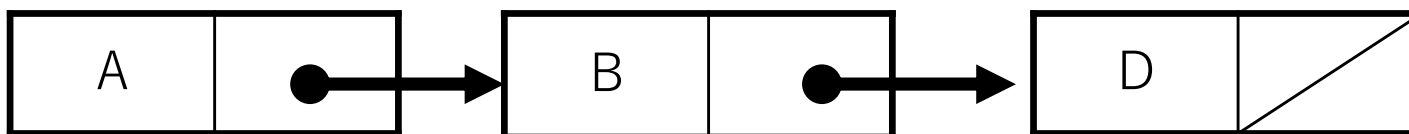
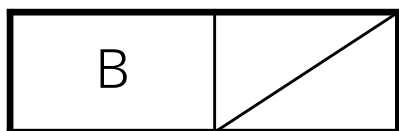
- リストの分断 (split)
 - 指定された要素の次の位置からリストを分断する

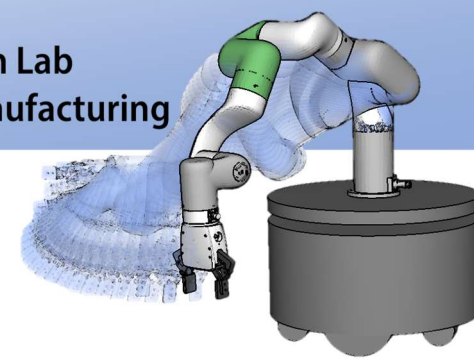




連結リストの操作

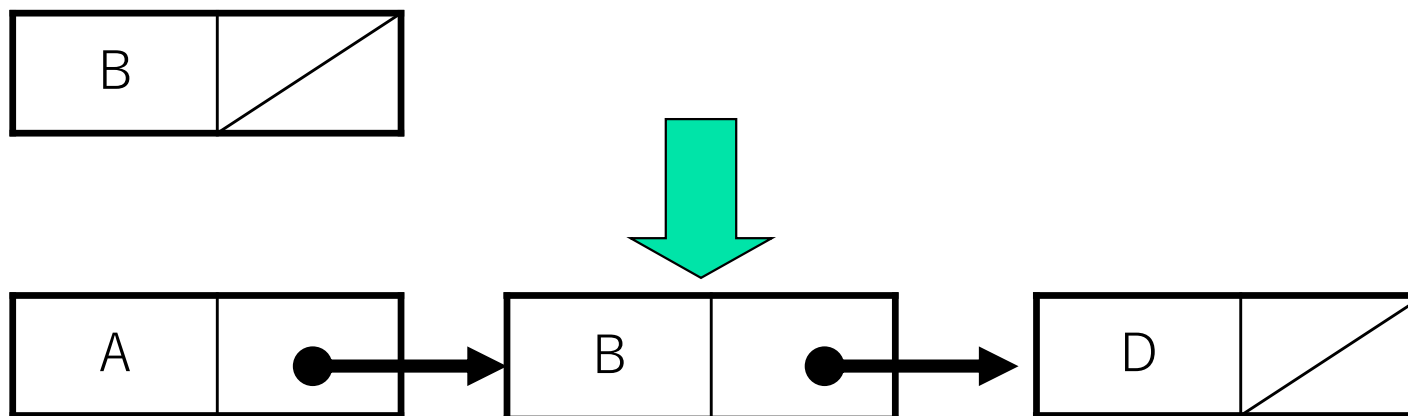
- 要素の探索 (find)
 - 指定された値を持つ要素の位置を返す

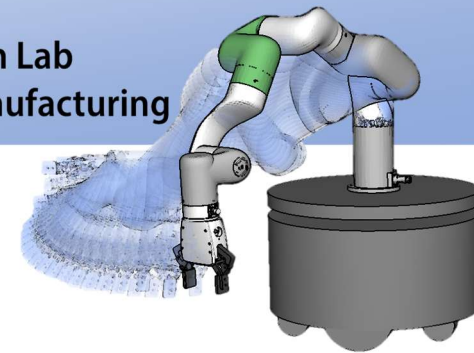




連結リストの操作

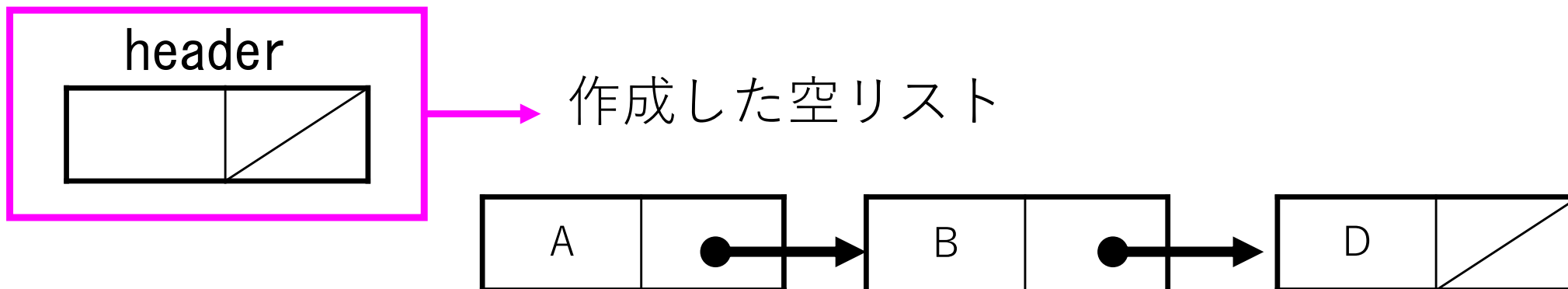
- 要素の探索 (find)
 - 指定された値を持つ要素の位置を返す

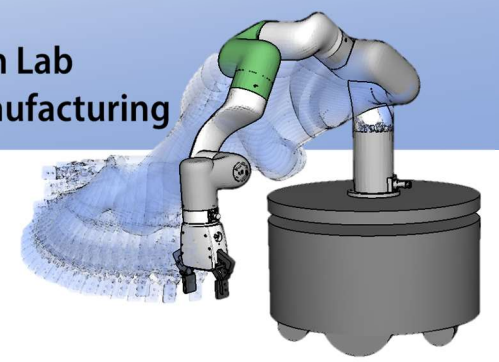




連結リストの操作

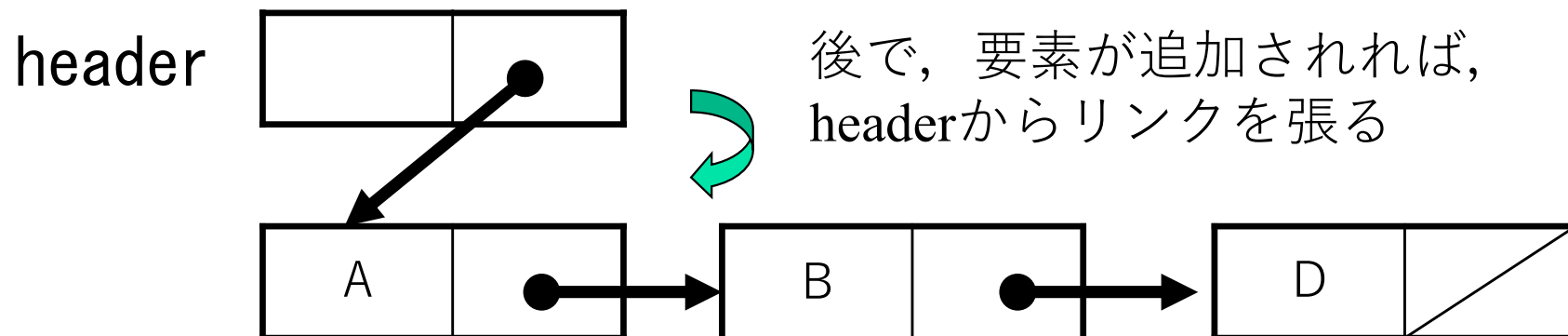
- 空リストの生成 (create)
 - 要素のないリスト
 - なぜ必要か？
 - 要素をすべて削除する可能性がある
 - どう表す？
 - ヘッダを付加して表現
 - ヘッダのみの場合は空リストとして扱う

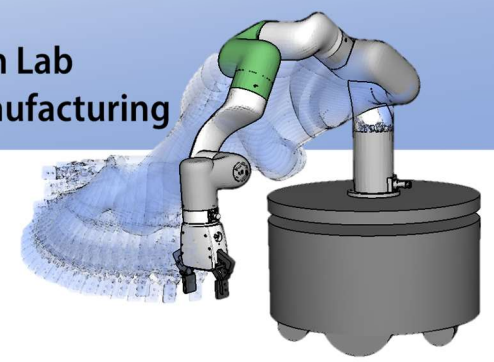




連結リストの操作

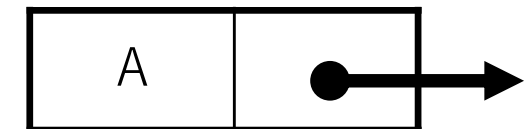
- 空リストの生成 (create)
 - 要素のないリスト
 - なぜ必要か？
 - 要素をすべて削除する可能性がある
 - どう表す？
 - ヘッダを付加して表現
 - ヘッダのみの場合は空リストとして扱う



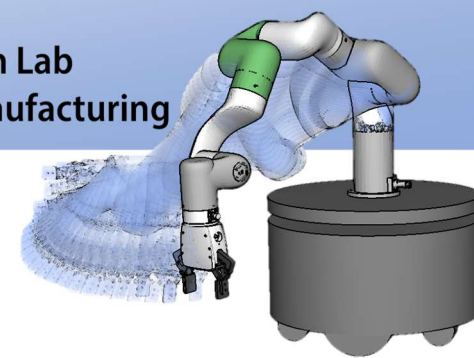


構造体とポインタを用いた 連結リストの実装

- 構造体を利用する利点
 - データと次のデータへのポインタが同時に管理できる
- ポインタを利用する利点
 - ?メモリ領域を利用して動的にメモリを使用できる
 - 固定のサイズに依存しない
 - 要素の位置指定に利用できる



```
struct CELL {  
    DATATYPE data; /* 格納されるデータ */  
    struct CELL *next; /* 次のデータへのポインタ */  
};
```



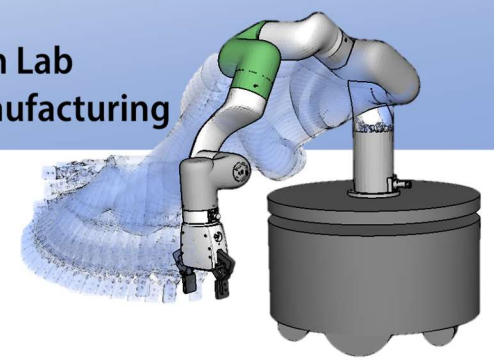
構造体とポインタを用いた 連結リストの実装

- 空リストの生成 (create)
 - ヘッダを作成して初期化する

```
struct CELL myList; /* List を表すヘッダ変数 */  
myList.next = NULL; /* 空を示す */  
myList.data = 0; /* header の data は使用されないが、  
一応 int 型として話を進める */
```



リストの生成はよく使う & 要素の生成にも利用できる
ので関数にする



構造体とポインタを用いた 連結リストの実装

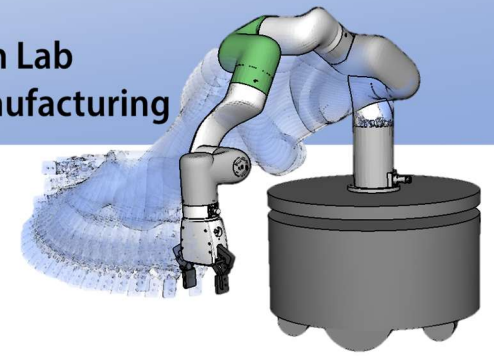
- 空リスト(要素)の生成 (create)
 - ヘッダを作成して初期化する
 - 作成したヘッダをポインタとして返す

```
struct CELL *create(DATATYPE data) {  
    struct CELL *p = (struct CELL*)malloc(sizeof(struct CELL));  
    p->next = NULL;  
    p->data = data;  
    return p;  
}
```

struct CELL の格納領域を
ヒープ領域に確保する

利用方法

```
struct CELL *header;  
header=create(data);
```



構造体とポインタを用いた 連結リストの実装

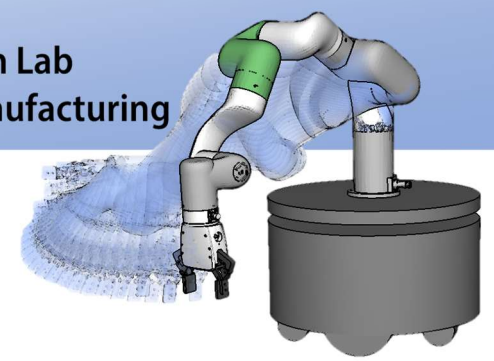
```
struct CELL *create(DATATYPE data) {  
    struct CELL *p = (struct CELL*)malloc(sizeof(struct CELL));  
    p->next = NULL;  
    p->data = data;  
    return p;  
}
```

struct CELL の格納領域を
ヒープ領域に確保する

間違ったコーディング例)

```
struct CELL *create(DATATYPE data) {  
    struct CELL p; /* 局所変数 */  
    p.next = NULL;  
    p.data = data;  
    return &p;  
}
```

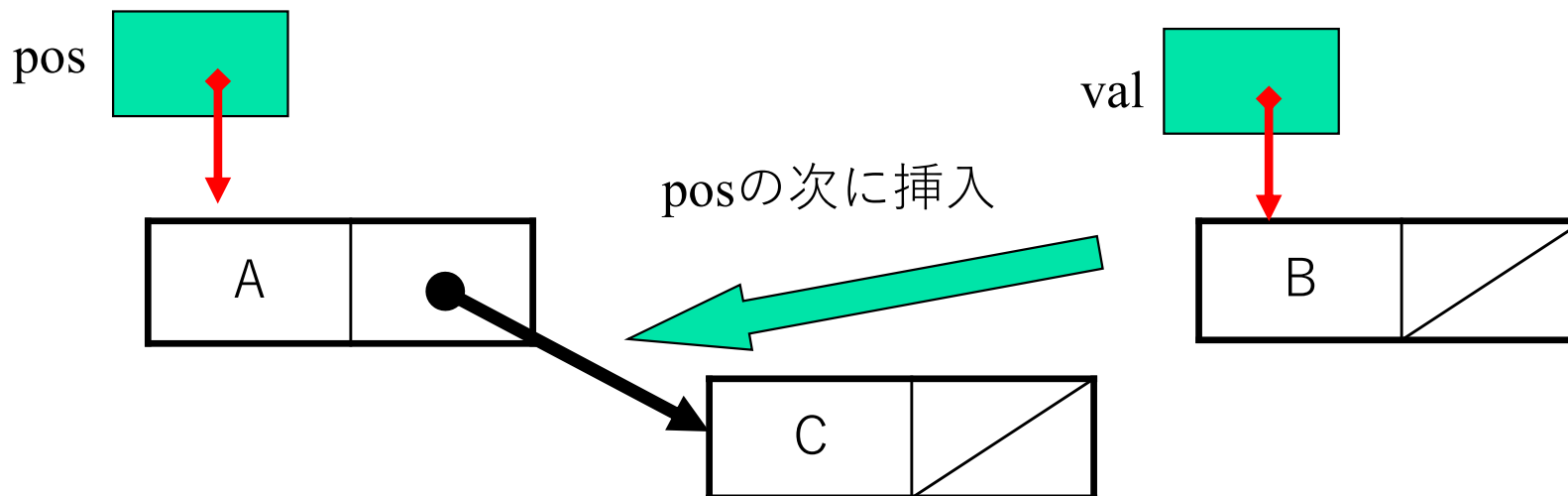
```
struct CELL create(DATATYPE data) {  
    struct CELL p; /* 局所変数 */  
    p.next = NULL;  
    p.data = data;  
    return p;  
}
```

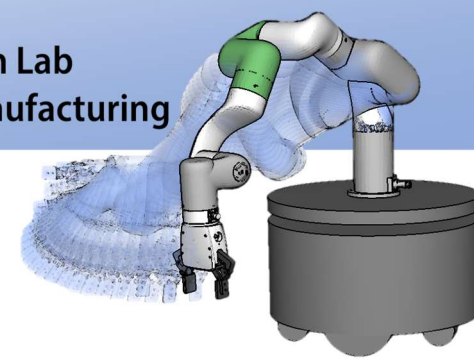


要素挿入の実装

- 指定された要素の次の位置に挿入する
 - 要素の指定にはポインタを利用する
 - 挿入する要素もポインタで指定する

```
void insert( struct CELL *pos, struct CELL *val ) {  
    val->next = pos->next;  
    pos->next = val;  
}
```

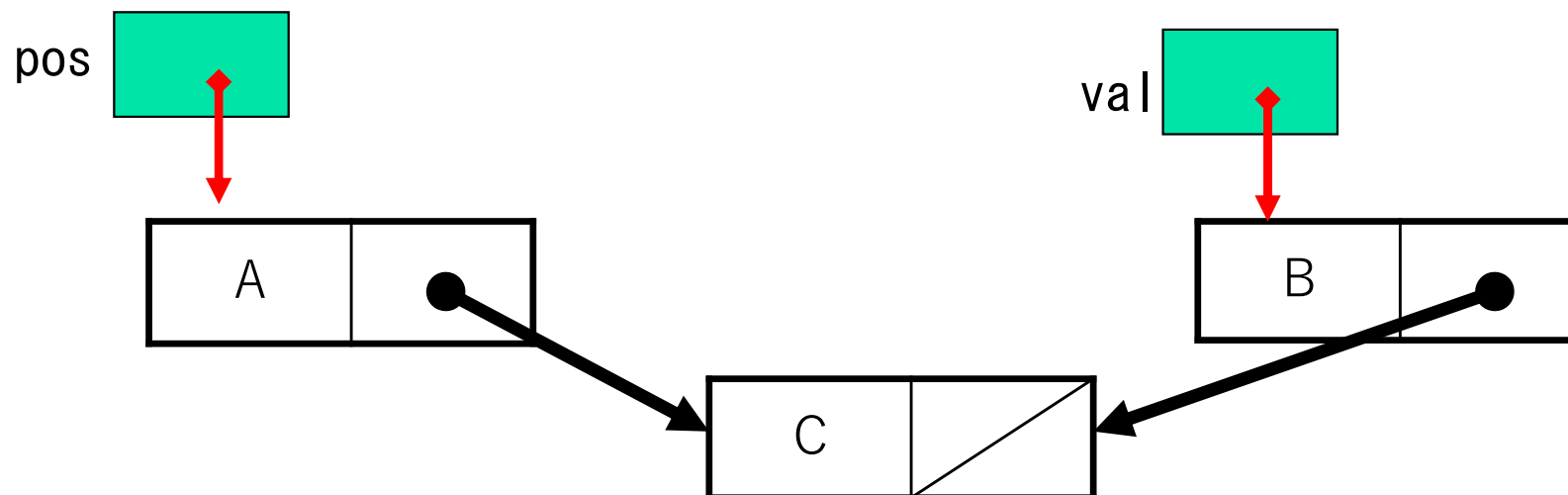


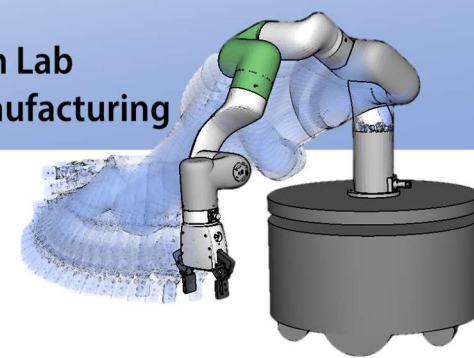


要素挿入の実装

- 指定された要素の次の位置に挿入する
 - 要素の指定にはポインタを利用する
 - 挿入する要素もポインタで指定する

```
void insert( struct CELL *pos, struct CELL *val ) {  
    val->next = pos->next;  
    pos->next = val;  
}
```

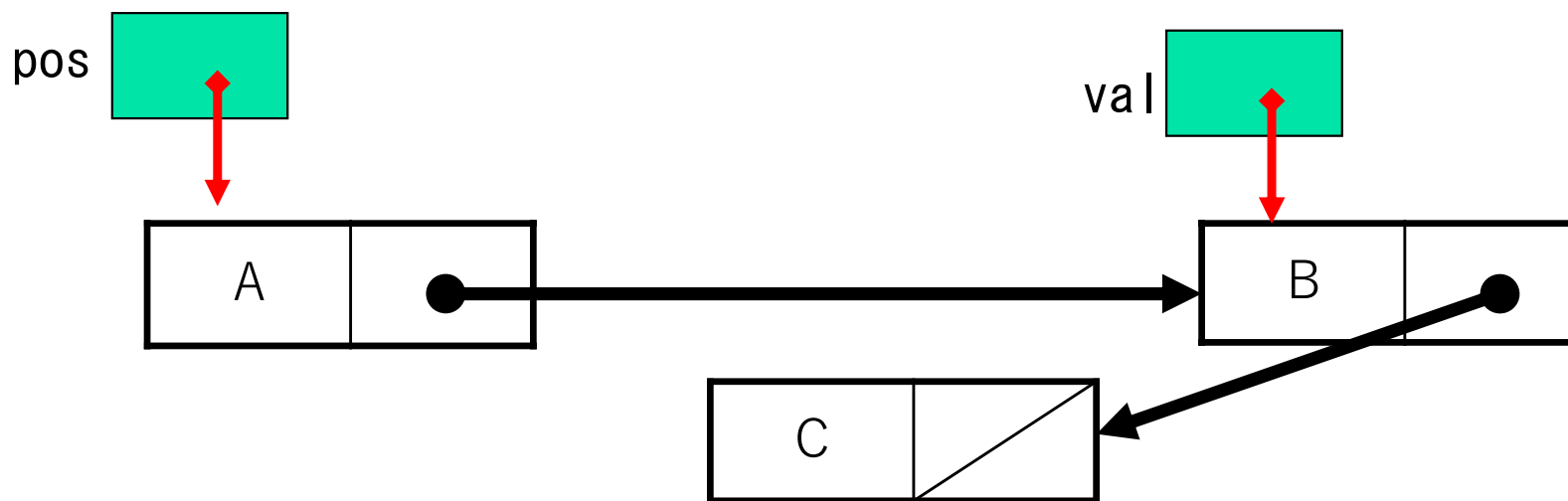


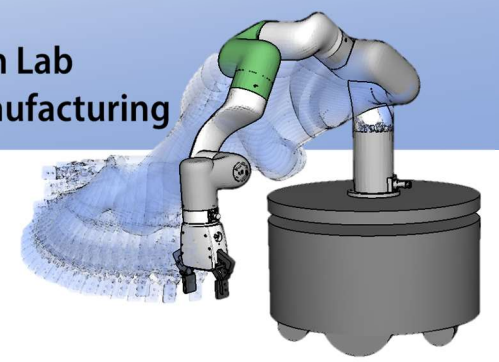


要素挿入の実装

- 指定された要素の次の位置に挿入する
 - 要素の指定にはポインタを利用する
 - 挿入する要素もポインタで指定する

```
void insert( struct CELL *pos, struct CELL *val ) {  
    val->next = pos->next;  
    pos->next = val;  
}
```

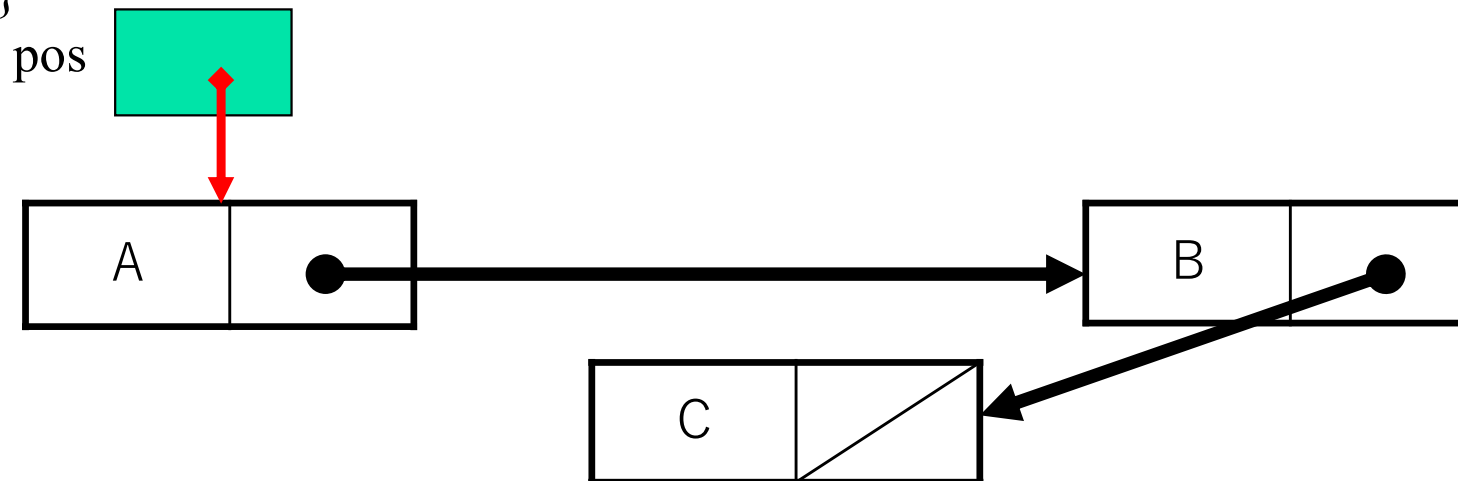


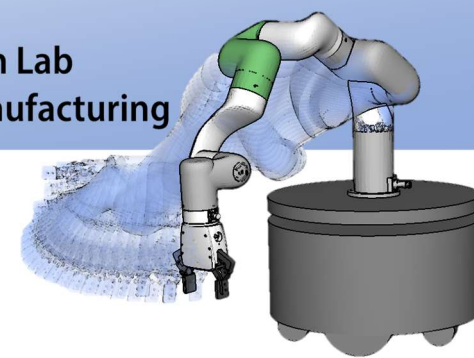


要素削除の実装

- 指定された要素の次を削除する
 - 要素の指定にはポインタを利用する
 - 削除された要素のポインタを返す

```
struct CELL *erase( struct CELL *pos ) {  
    struct CELL *p = pos->next;  
    if (p != NULL) { pos->next = p->next; p->next = NULL; }  
    return p;  
}
```

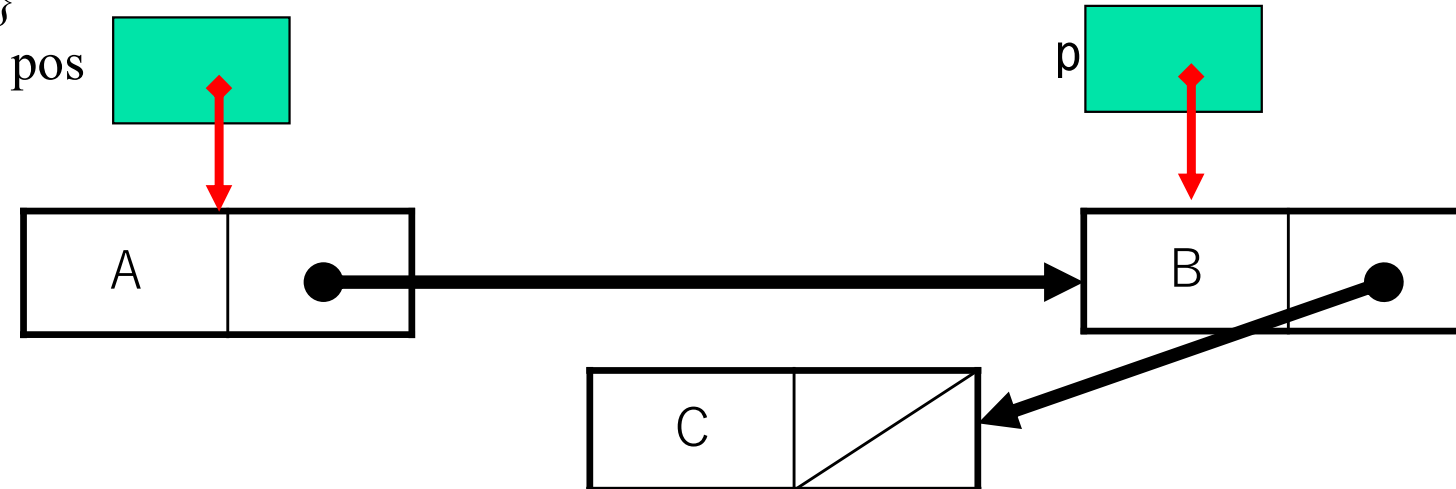


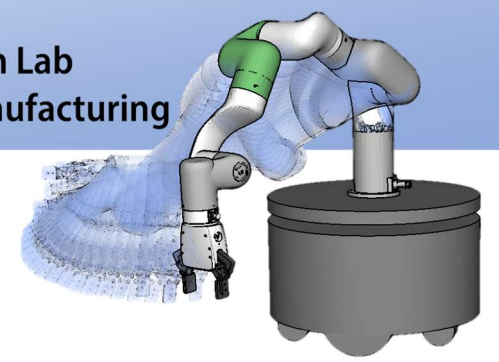


要素削除の実装

- 指定された要素の次を削除する
 - 要素の指定にはポインタを利用する
 - 削除された要素のポインタを返す

```
struct CELL *erase( struct CELL *pos ) {  
    struct CELL *p = pos->next;  
    if (p != NULL) { pos->next = p->next; p->next = NULL; }  
    return p;  
}
```

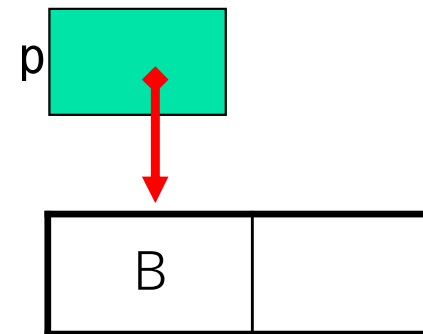
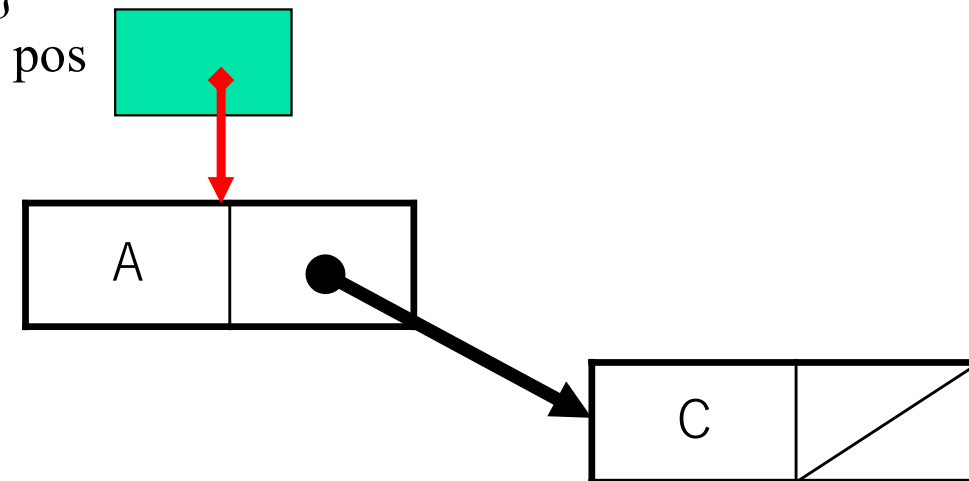


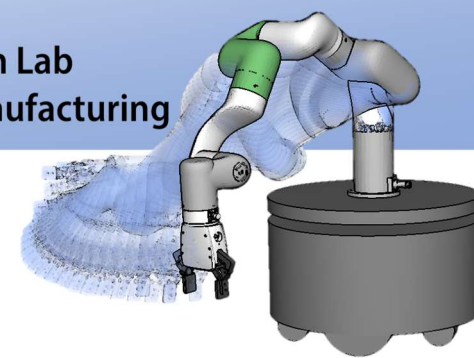


要素削除の実装

- 指定された要素の次を削除する
 - 要素の指定にはポインタを利用する
 - 削除された要素のポインタを返す

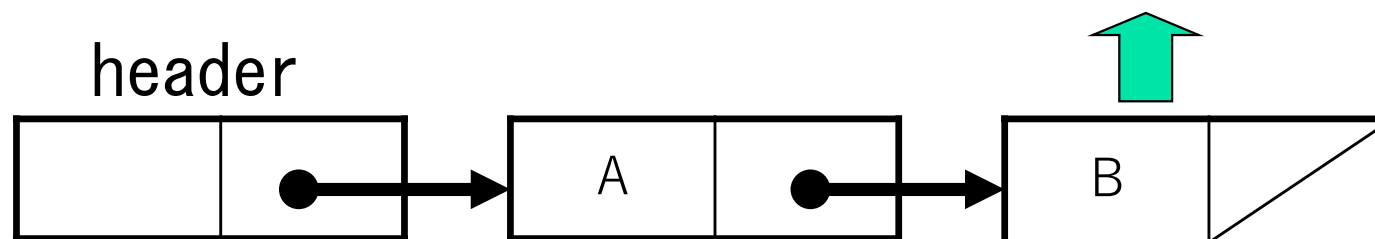
```
struct CELL *erase( struct CELL *pos ) {  
    struct CELL *p = pos->next;  
    if (p != NULL) { pos->next = p->next; p->next = NULL; }  
    return p;  
}
```

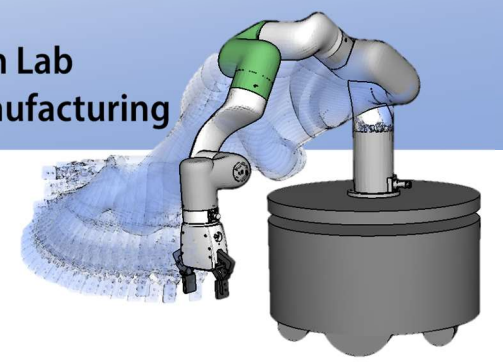




最後尾要素を見る(tail)

- リストの最後尾を返す

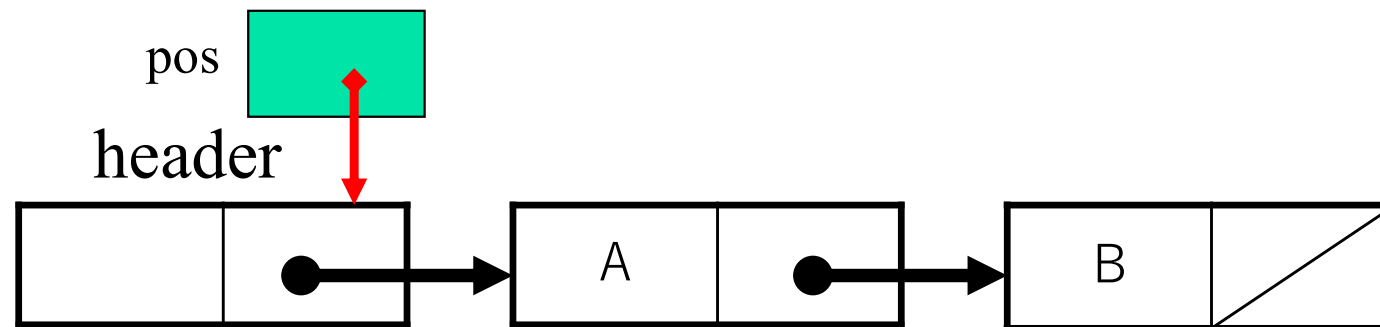


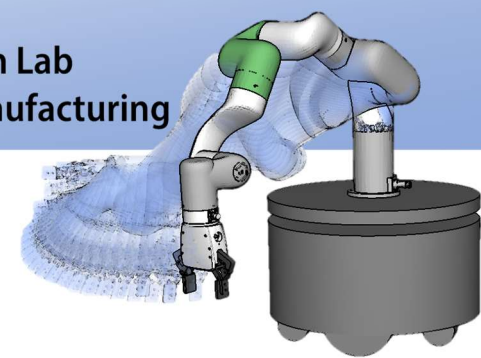


tail の実装

- リストの最後尾を返す

```
struct CELL *tail( struct CELL *pos ) {  
    while (pos->next != NULL) pos = pos->next;  
    return pos;  
}
```

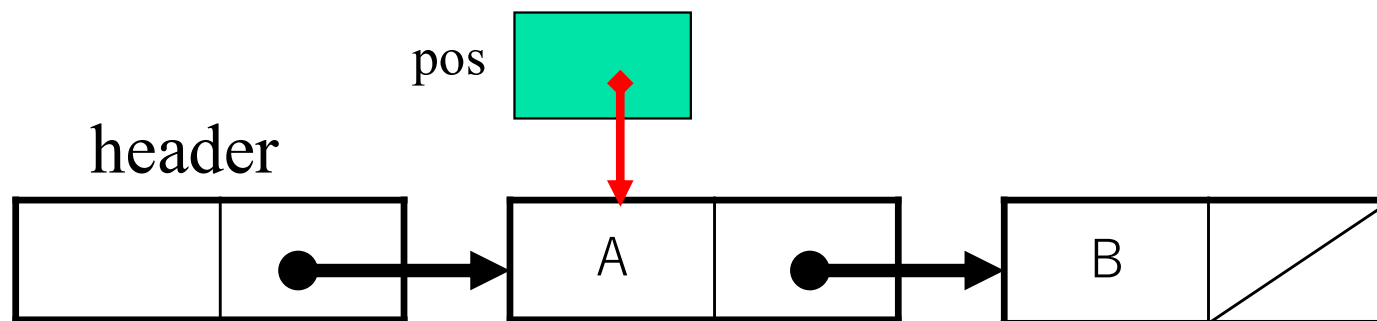


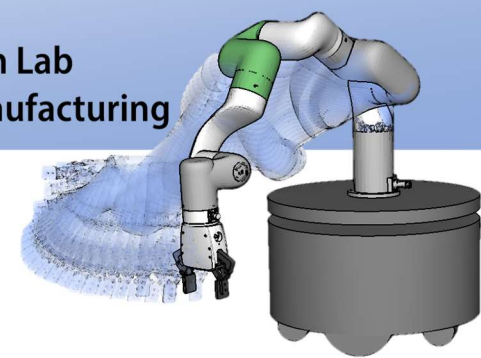


tail の実装

- リストの最後尾を返す

```
struct CELL *tail( struct CELL *pos ) {  
    while (pos->next != NULL) pos = pos->next;  
    return pos;  
}
```

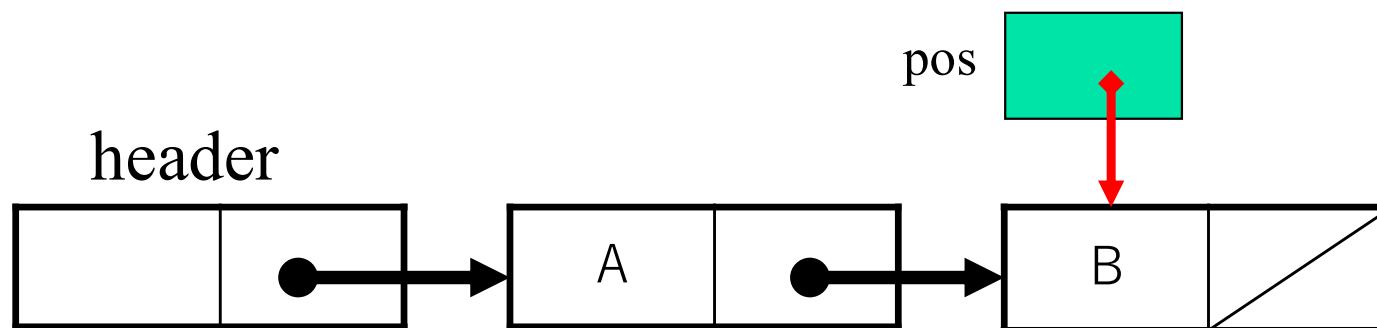


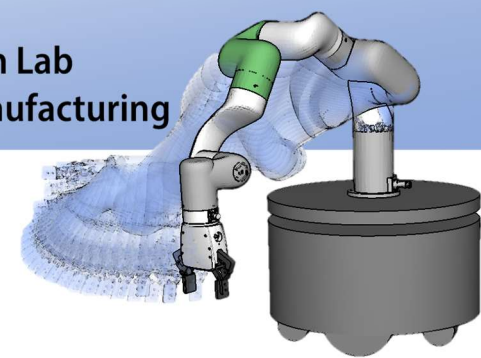


tail の実装

- リストの最後尾を返す

```
struct CELL *tail( struct CELL *pos ) {  
    while (pos->next != NULL) pos = pos->next;  
    return pos;  
}
```

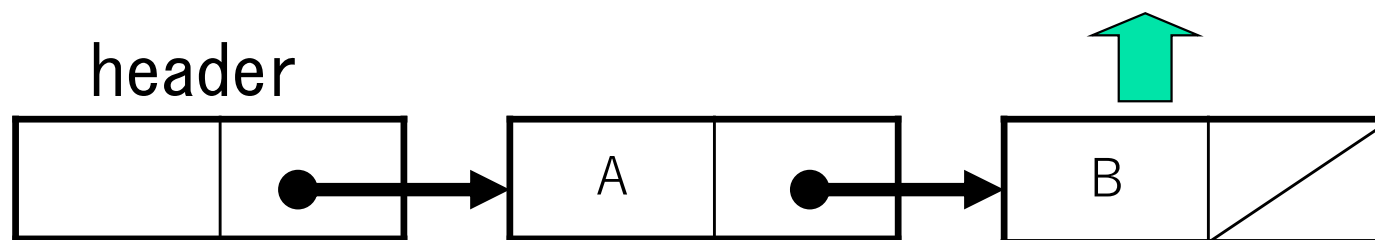


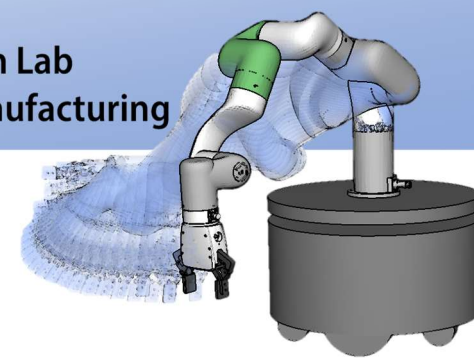


tail の実装

- リストの最後尾を返す

```
struct CELL *tail( struct CELL *pos ) {  
    while (pos->next != NULL) pos = pos->next;  
    return pos;  
}
```

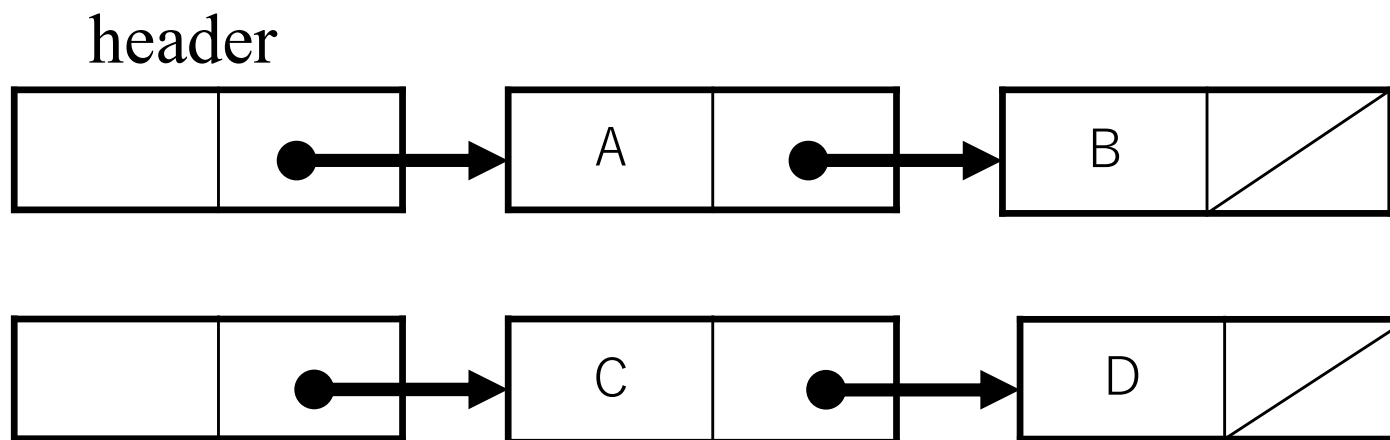


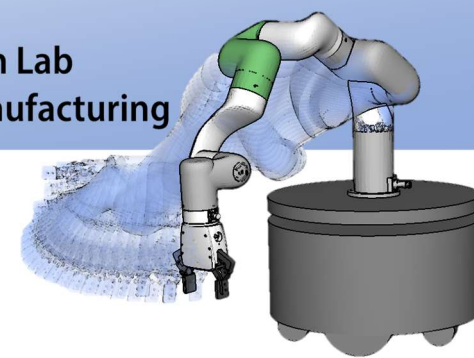


リスト連結の実装

- 二つのリストのヘッダを渡す
- 連結されたヘッダを返す

```
struct CELL *concatenate( struct CELL *L1, struct CELL *L2 );
```



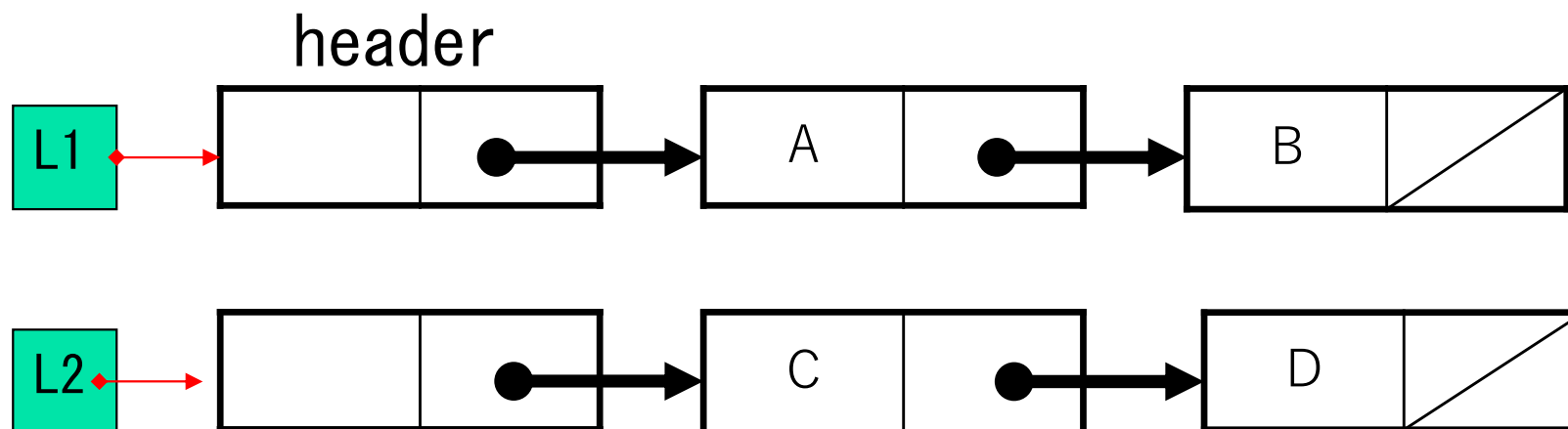


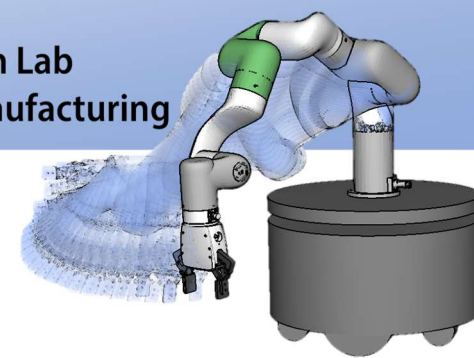
リスト連結の実装

- 二つのリストのヘッダを渡す
- 連結されたヘッダを返す

```

struct CELL *concatenate( struct CELL *L1, struct CELL *L2 ) {
    struct CELL *p = tail( L1 );
    p->next = L2->next; L2->next = NULL;
    return L1;
};
  
```



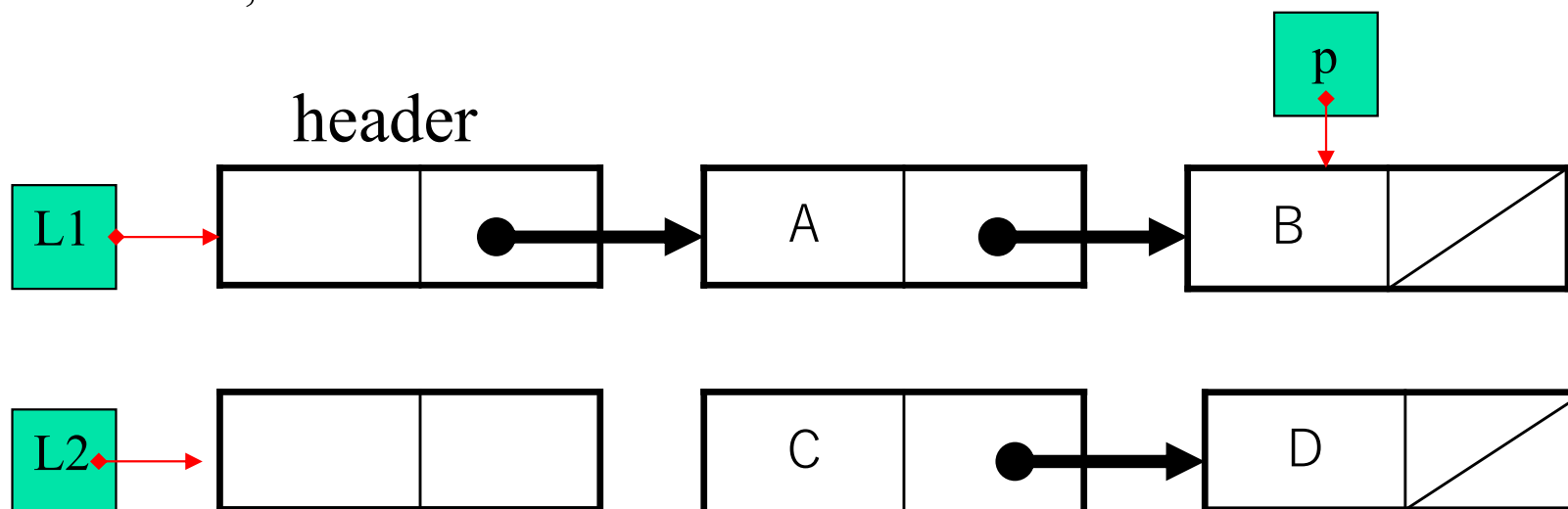


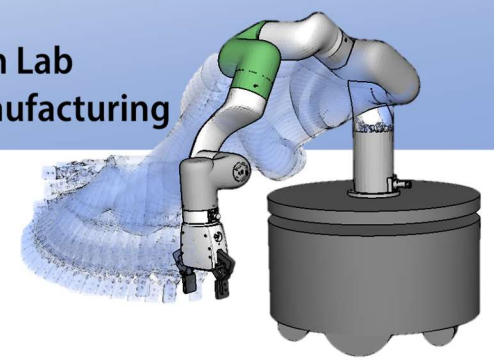
リスト連結の実装

- 二つのリストのヘッダを渡す
- 連結されたヘッダを返す

```

struct CELL *concatenate( struct CELL *L1, struct CELL *L2 ) {
    struct CELL *p = tail( L1 );
    p->next = L2->next; L2->next = NULL;
    return L1;
};
  
```



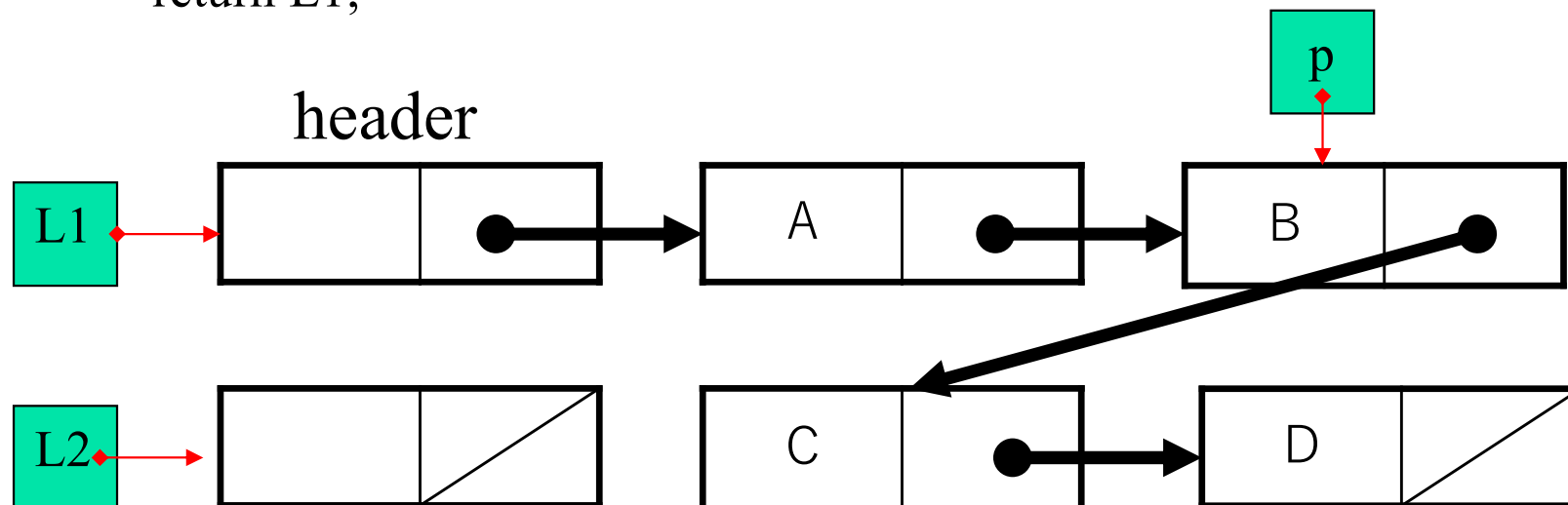


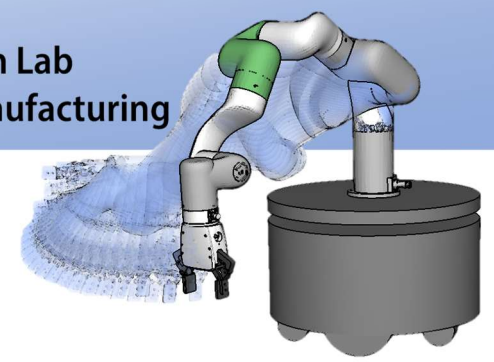
リスト連結の実装

- 二つのリストのヘッダを渡す
- 連結されたヘッダを返す

```

struct CELL *concatenate( struct CELL *L1, struct CELL *L2 ) {
    struct CELL *p = tail( L1 );
    p->next = L2->next; L2->next = NULL;
    return L1;
};
  
```



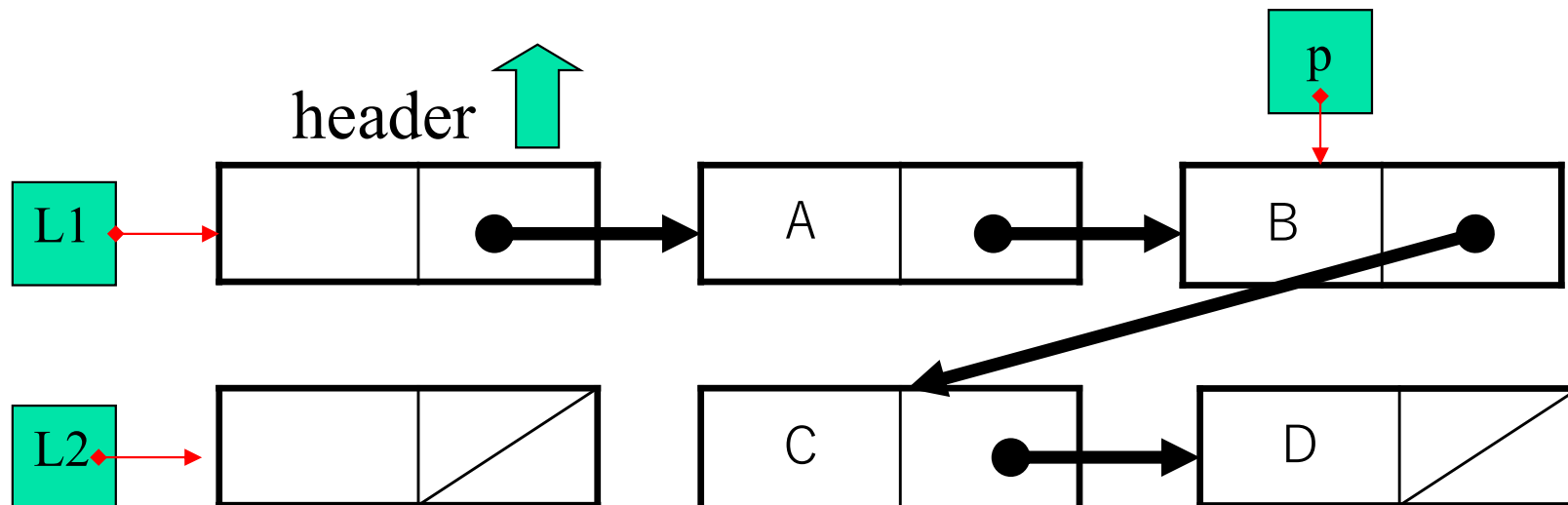


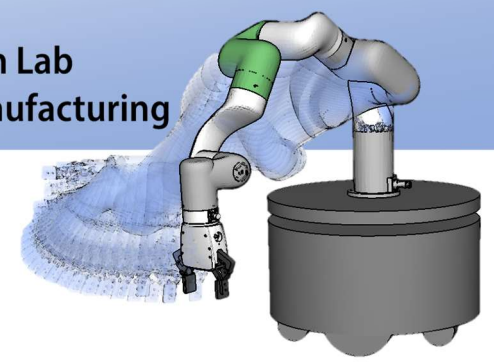
リスト連結の実装

- 二つのリストのヘッダを渡す
- 連結されたヘッダを返す

```

struct CELL *concatenate( struct CELL *L1, struct CELL *L2 ) {
    struct CELL *p = tail( L1 );
    p->next = L2->next; L2->next = NULL;
    return L1;
};
  
```

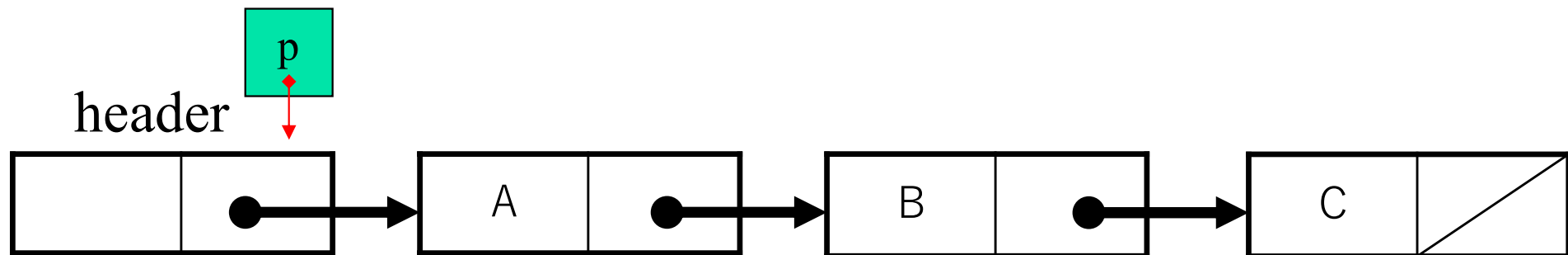


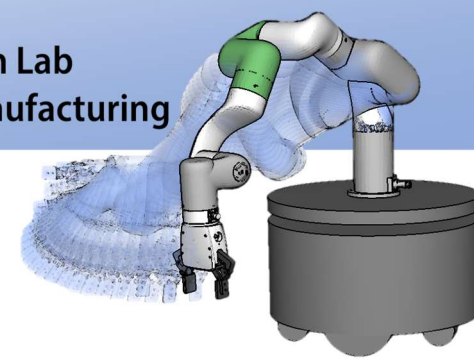


要素探索の実装

- 要素を比較し等価な要素を探索
- 最初に見つけた要素の位置を返す
- なければ NULL を返す

```
struct CELL *find( struct CELL *p, DATATYPE val );
```

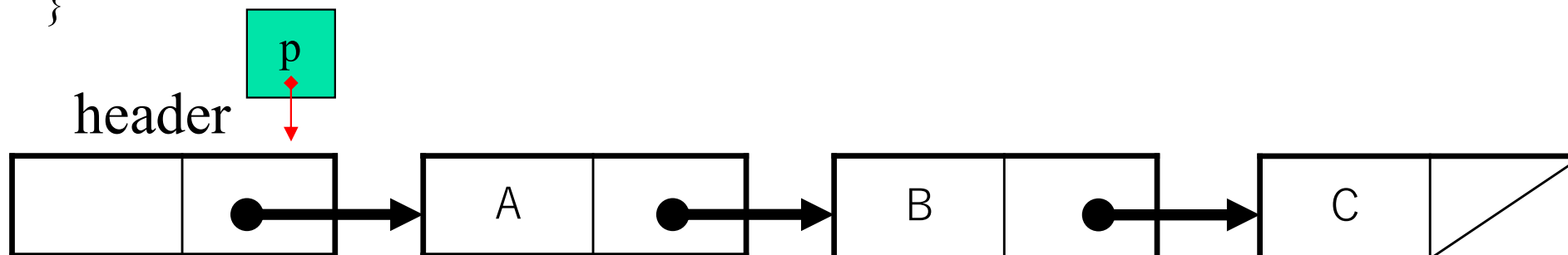


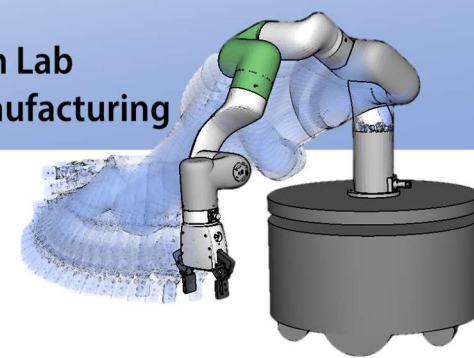


要素探索の実装

- 要素を比較し等価な要素を探索
- 最初に見つけた要素の位置を返す
- なければ NULL を返す

```
struct CELL *find( struct CELL *p, DATATYPE val ) {  
    p = p->next;  
    while (p != NULL)  
        if (isEqual(p->data, val)) return p; else p = p->next;  
    return p;  
}
```

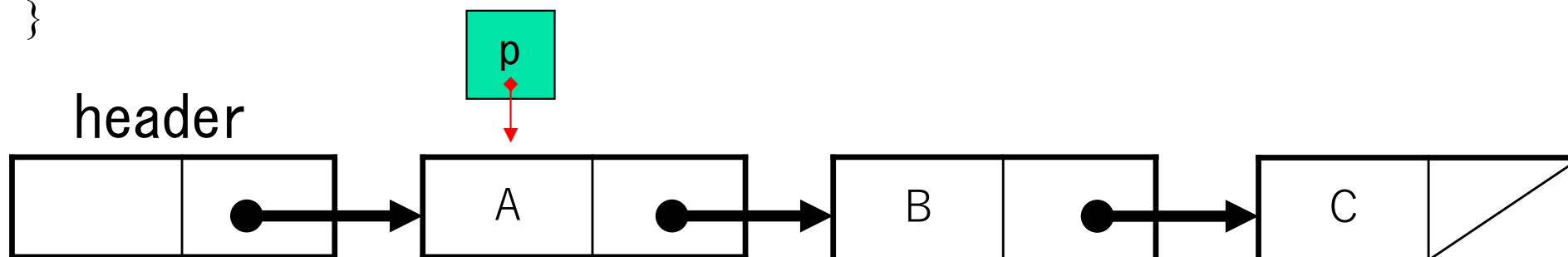


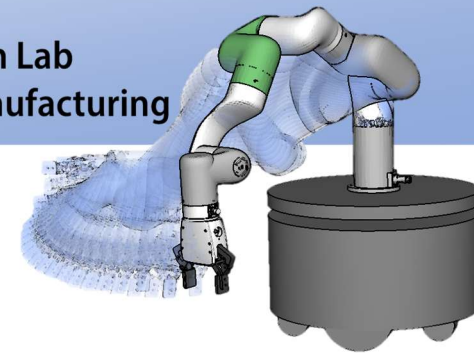


要素探索の実装

- 要素を比較し等価な要素を探索
- 最初に見つけた要素の位置を返す
- なければ NULL を返す

```
struct CELL *find( struct CELL *p, DATATYPE val ) {  
    p = p->next;  
    while (p != NULL)  
        if (isEqual(p->data, val)) return p; else p = p->next;  
    return p;  
}
```

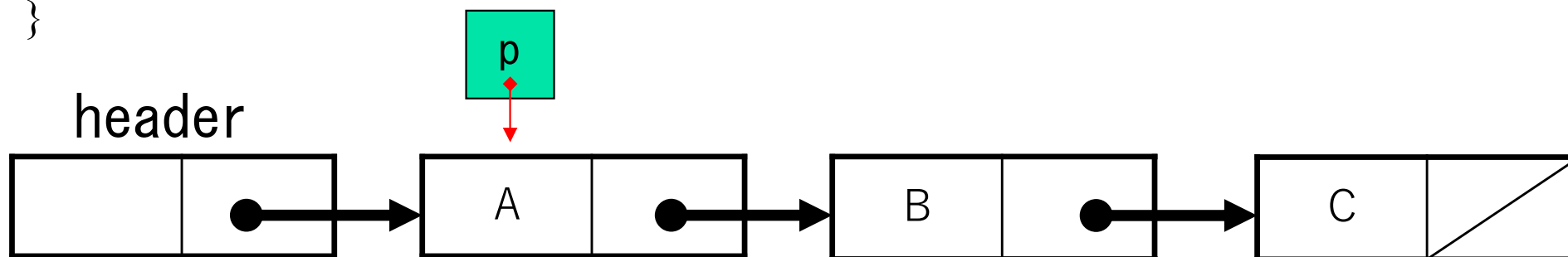


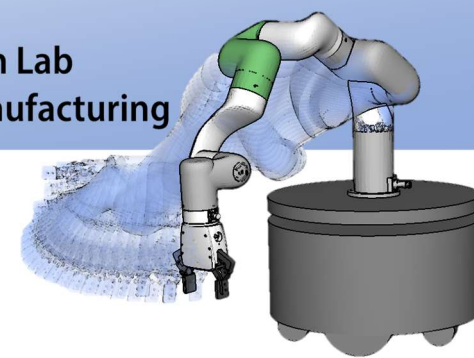


要素探索の実装

- 要素を比較し等価な要素を探索
- 最初に見つけた要素の位置を返す
- なければ NULL を返す

```
struct CELL *find( struct CELL *p, DATATYPE val ) {  
    p = p->next;  
    while (p != NULL)  
        if (isEqual(p->data, val)) return p; else p = p->next;  
    return p;  
}
```

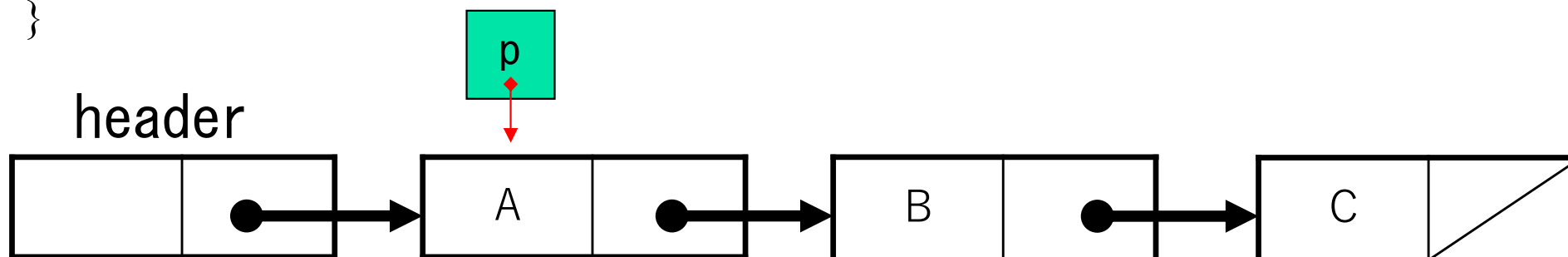


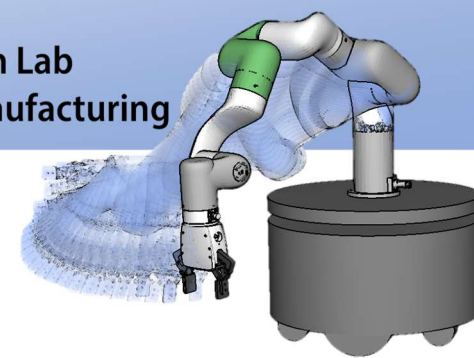


要素探索の実装

- 要素を比較し等価な要素を探索
- 最初に見つけた要素の位置を返す
- なければ NULL を返す

```
struct CELL *find( struct CELL *p, DATATYPE val ) {  
    p = p->next;  
    while (p != NULL)  
        if (isEqual(p->data, val)) return p; else p = p->next;  
    return p;  
}
```

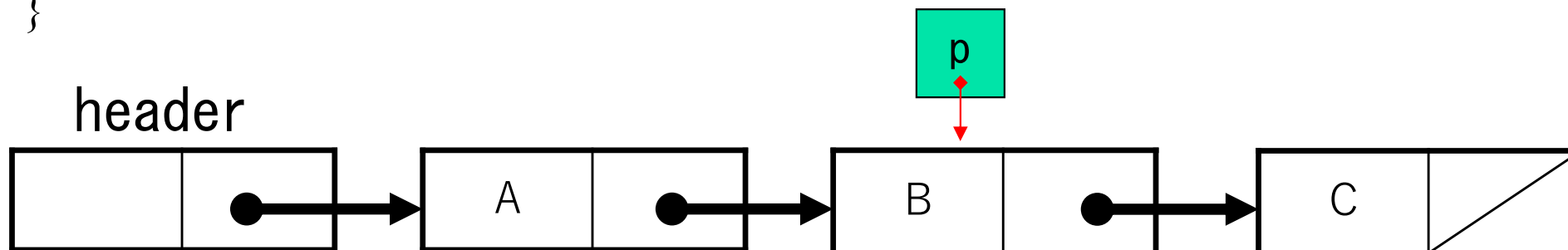


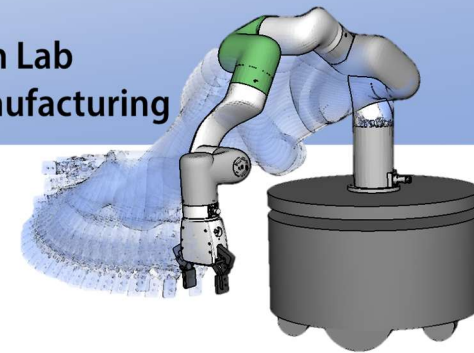


要素探索の実装

- 要素を比較し等価な要素を探索
- 最初に見つけた要素の位置を返す
- なければ NULL を返す

```
struct CELL *find( struct CELL *p, DATATYPE val ) {  
    p = p->next;  
    while (p != NULL)  
        if (isEqual(p->data, val)) return p; else p = p->next;  
    return p;  
}
```

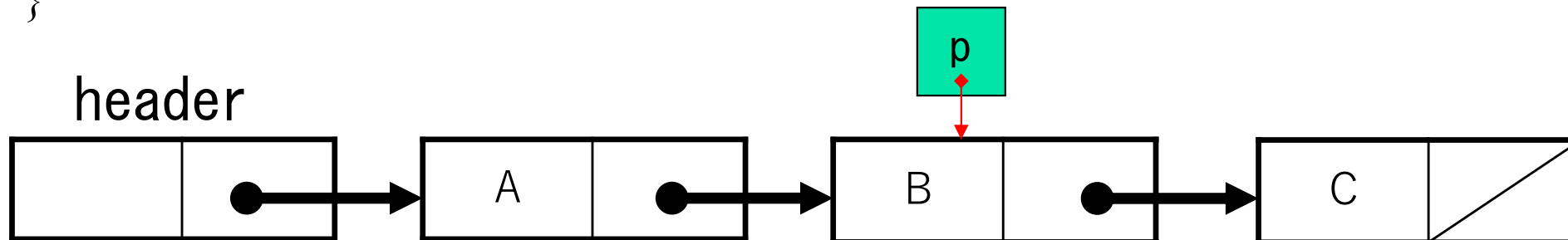


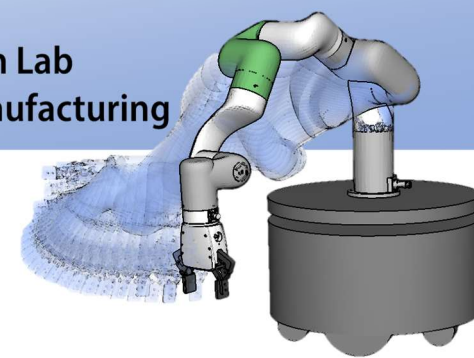


要素探索の実装

- 要素を比較し等価な要素を探索
- 最初に見つけた要素の位置を返す
- なければ NULL を返す

```
struct CELL *find( struct CELL *p, DATATYPE val ) {  
    p = p->next;  
    while (p != NULL)  
        if (isEqual(p->data, val)) return p; else p = p->next;  
    return p;  
}
```

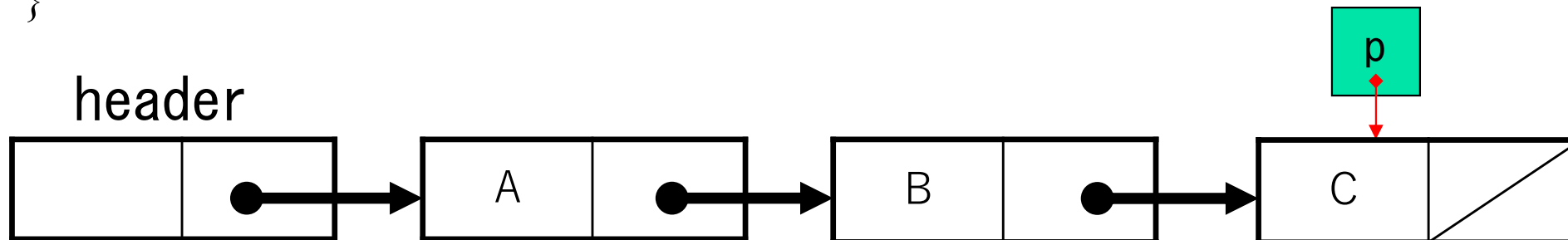


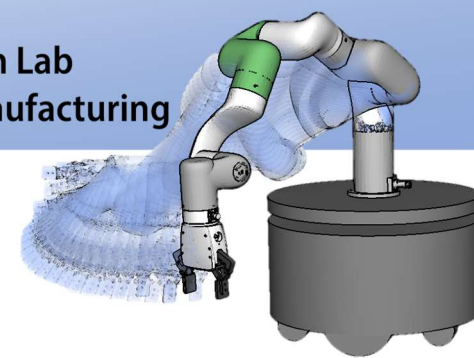


要素探索の実装

- 要素を比較し等価な要素を探索
- 最初に見つけた要素の位置を返す
- なければ NULL を返す

```
struct CELL *find( struct CELL *p, DATATYPE val ) {  
    p = p->next;  
    while (p != NULL)  
        if (isEqual(p->data, val)) return p; else p = p->next;  
    return p;  
}
```

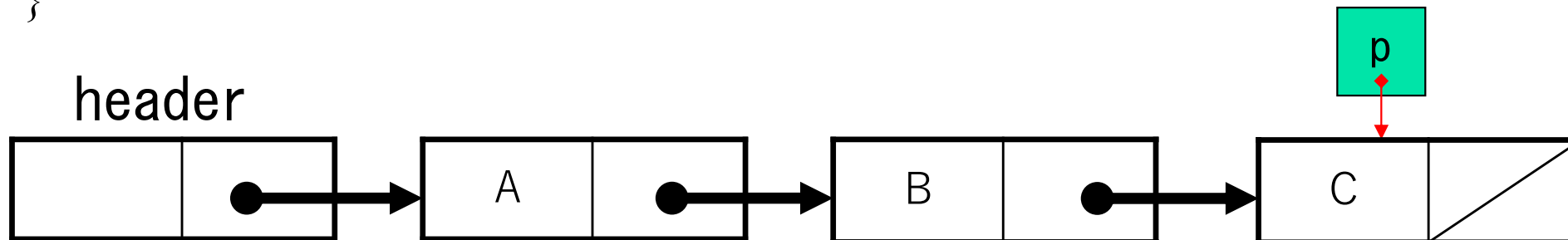


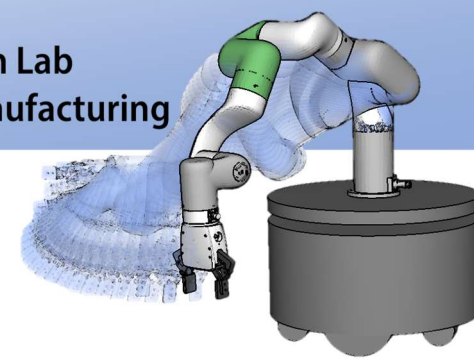


要素探索の実装

- 要素を比較し等価な要素を探索
- 最初に見つけた要素の位置を返す
- なければ NULL を返す

```
struct CELL *find( struct CELL *p, DATATYPE val ) {  
    p = p->next;  
    while (p != NULL)  
        if (isEqual(p->data, val)) return p; else p = p->next;  
    return p;  
}
```

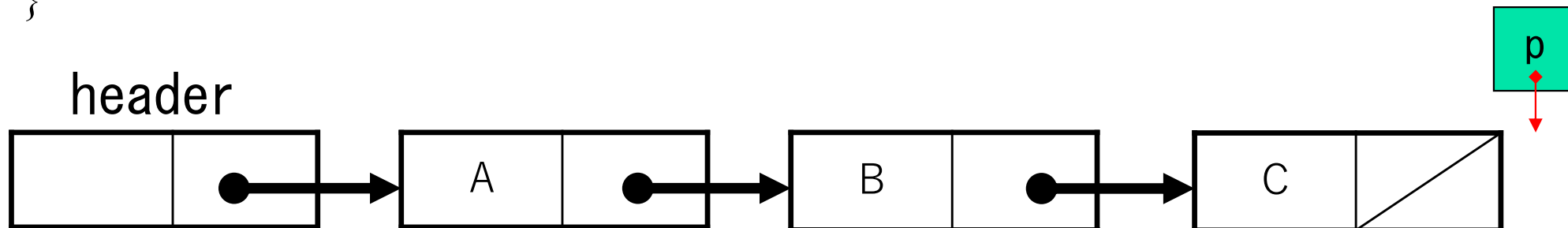


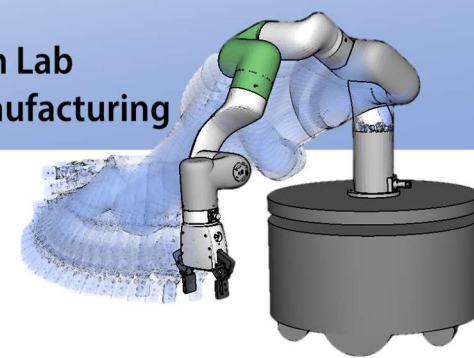


要素探索の実装

- 要素を比較し等価な要素を探索
- 最初に見つけた要素の位置を返す
- なければ NULL を返す

```
struct CELL *find( struct CELL *p, DATATYPE val ) {  
    p = p->next;  
    while (p != NULL)  
        if (isEqual(p->data, val)) return p; else p = p->next;  
    return p;  
}
```





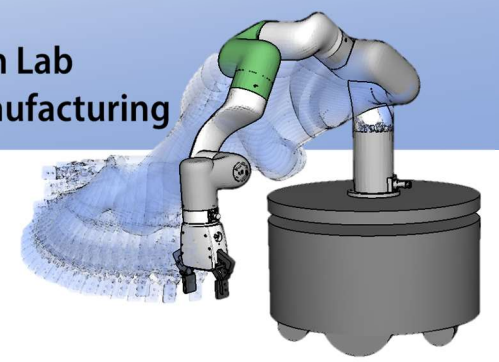
等価判定関数 isEqual の実装

- 等価判定
 - 二つのデータが等しいかどうか
 - 演算子=は基本データ型にしか利用できない
- DATATYPE の中身で等価判定関数は異なる
(実装, 設計により異なる)
- 等価とはどういうことか定義が必要

```
int isEqual(DATATYPE d1, DATATYPE d2);
```

d1 と d2 が等価ならば !0

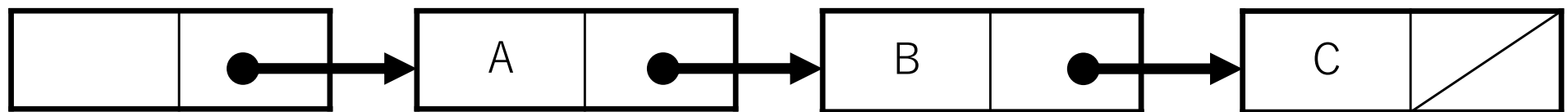
そうでないならば 0 を返す



循環リスト

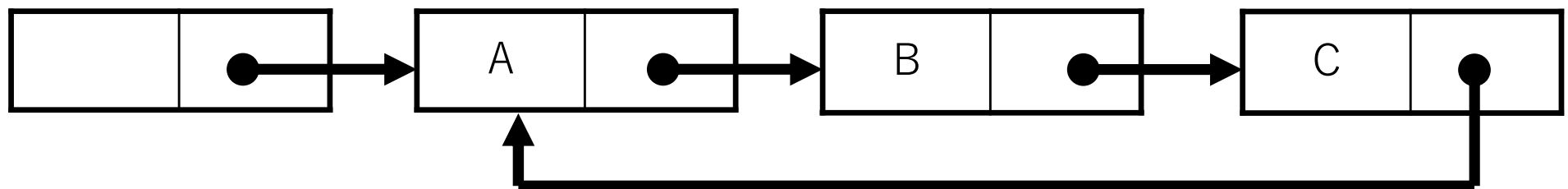
連結リスト linked list

header

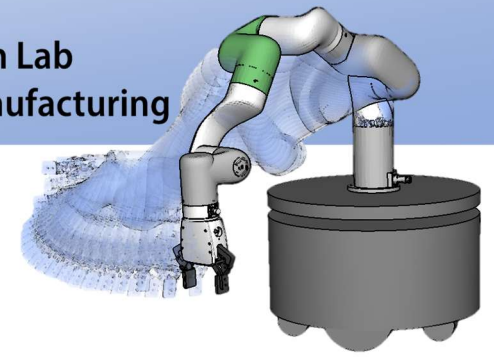


循環リスト circular list

header

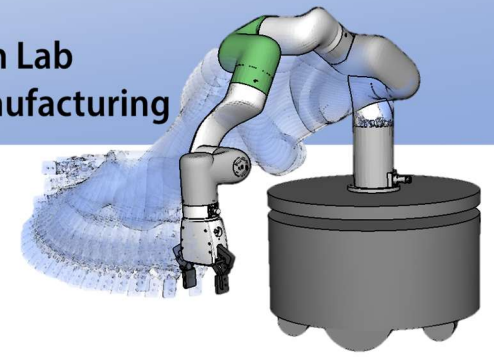


後ろに移動し続ければ、先頭に戻る



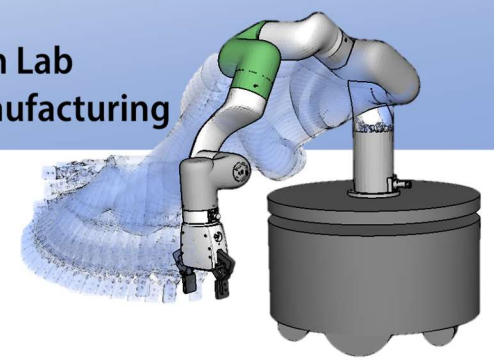
循環リストの特徴と目的

- 環状に並んだデータ構造を表現
- 要素を結び合わせるポインタをたどれば, 循環リストに含まれる全要素を順番に処理できる
- 厳密には「最初」と「最後」の要素というものは存在しない



循環リストの操作

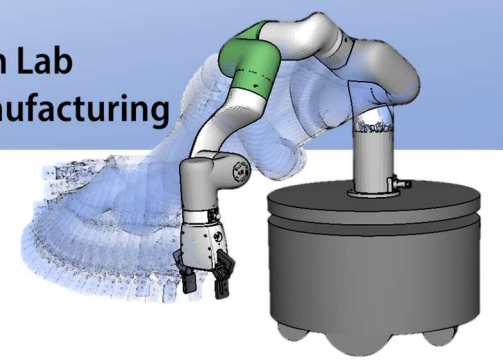
- 要素の挿入・削除
 - 連結リストの場合と同様
- 要素の追跡(trace)
 - 基本的に連結リストと同様
 - 最後尾がNULLでは判定できないので工夫が必要→追跡を開始した要素に到達したら終了



要素追跡の実装例

```
struct CELL *p, *ptr; /* 追跡用のポインタ変数 */  
  
if (ptr != NULL) { /* 追跡開始点 ptr */  
    p = ptr; /* 追跡開始点を作業用変数 p にコピー */  
    do {  
        /* p が指すセルの処理 */  
        p = p->next;  
    } while (p != ptr); /* 開始点まで戻ってきたら終了 */  
}
```

変数 ptr が NULL なら，この循環リストは空なので何もしない。
リストが空でないなら，セルを順番にたどって処理する
Do .. While 形式によりループ末尾で終了条件を判断する



まとめ

- スタック
 - LIFO
- キュー
 - FIFO
- 再帰
 - 繰り返し作業で効率化
- リスト